Projekt: Software Engineering

(DLMCSPSE01_D)

IU Internationale Hochschule

Studiengang: Informatik

Projektdokumentation

Jessica Krautheuser

Matrikelnummer: IU14129800

Tutor: Prof. Dr. Dirk Simon

Abgabedatum: 30.10.2026

Inhalt

Tabellenverzeichnis III

Abkürzungsverzeichnis IV

1. Projektidee 1

2. Projektmotivation 1

3. Projektmethodik 1

4. Projektübersicht 3

4.1 Projektziel 3

4.2 Spielprinzip 3

4.3 Zielgruppe 4

4.4 Kundennutzen 4

4.5 Projektumfang 4

5. Risikomanagement 6

6. Zeitplanung 8

Literaturverzeichnis IV

Tabellenverzeichnis

Tabelle 1: Risikomatrix 6
Tabelle 2: Arbeitspakete 8
Tabelle 3: Zeitplan 9
Tabelle 4: Meilensteine 9

Abkürzungsverzeichnis

A	Auswirkung
AP	Arbeitspaket
E	Eintrittswahrscheinlichkeit
GUI	Graphic User Interface
M	Meilenstein

1. Projektidee

Im Rahmen des Projekts wird ein Spiel mit dem Titel Mystic Maze entwickelt. Das grundlegende Spielprinzip orientiert sich an dem Gesellschaftsspiel „Das verrückte Labyrinth“ des Ravensburger Verlags (Ravensburger AG, o. J.). Dabei wird ein aus einzelnen Kacheln bestehendes Spielfeld während des Spielverlaufs dynamisch verändert. Die Spieler müssen das Labyrinth gezielt verschieben, um Wege zu bestimmten Zielen zu öffnen und gleichzeitig die Möglichkeiten der Mitspielenden einzuschränken.

Im Gegensatz zum grundlegenden Spielprinzip von „Das verrückte Labyrinth“ wird Mystic Maze insbesondere durch Spezialkacheln sowie individuelle Charakterfähigkeiten erweitert. Dadurch entstehen zusätzliche taktische Entscheidungsmöglichkeiten und ein eigenständiges Spielkonzept.

Die Anwendung wird in Python entwickelt und als Webanwendung bereitgestellt.

Der Quellcode und die zugehörigen Dateien des Projekts werden in einem öffentlichen GitHub-Repository bereitgestellt. Dieses ist unter https://github.com/mysticmazejk/Mystic_Maze abrufbar.

2. Projektmotivation

Die Motivation für Mystic Maze ergibt sich aus dem persönlichen Interesse an strategischen Gesellschaftsspielen und dem Ziel, ein bereits bestehendes Spielprinzip durch eigene Ideen weiterzuentwickeln und dabei Kenntnisse des Software Engineerings praktisch anzuwenden. Die Umsetzung als browserbasierte Mehrspieler-Anwendung bietet dabei die Möglichkeit, unterschiedliche Herausforderungen wie komplexe Spiellogik, Client-Server-Kommunikation, Synchronisation, Testbarkeit und eine klare Trennung von Logik und Benutzeroberfläche praktisch zu bearbeiten. Gleichzeitig können zentrale Methoden des Requirements Engineering, der UML-Modellierung, der Softwarearchitektur und der Qualitätssicherung in einem zusammenhängenden Projekt angewendet werden.

Aus der Projektidee und der Projektmotivation ergibt sich folgende übergeordnete Projektfrage:

Wie kann das beschriebene Spiel Mystic Maze mithilfe geeigneter Methoden des Software Engineerings als wartbare, testbare und mehrspielerfähige Webanwendung konzipiert und umgesetzt werden?

3. Projektmethodik

Für die Entwicklung von Mystic Maze werden sowohl klassische als auch agile Methoden des Software Engineerings verwendet. Klassische Vorgehensweisen zeichnen sich durch eine strukturierte Planung, definierte Entwicklungsaktivitäten und eine vergleichsweise umfangreiche Dokumentation aus. Agile Vorgehensweisen setzen dagegen stärker auf iterative Entwicklung, regelmäßiges

Feedback und die Anpassung an neue Erkenntnisse (Mishra & Alzoubi, 2023, S. 1504-1506). Die Kombination beider Ansätze wird als hybrides Vorgehen bezeichnet. Dabei können strukturierte Vorgehensmodelle als Rahmen dienen und durch agile Praktiken ergänzt werden (Kuhrmann et al., 2017, S. 30-31) . Für Mystic Maze ermöglicht dieses Vorgehen, die Planungssicherheit der vorgegebenen Projektphasen mit der notwendigen Flexibilität während der technischen Entwicklung zu verbinden.

Das Projekt gliedert sich in die Konzeptionsphase, die Erarbeitungs- und Reflexionsphase sowie die Finalisierungsphase. In der Konzeptionsphase werden Projektplanung, Anforderungsermittlung und Spezifikation durchgeführt. Die Projektplanung definiert unter anderem Projektumfang, Arbeitspakete, Meilensteine, Risiken und Zeitplanung. Bei der Anforderungsanalyse werden anschließend die Anforderungen der relevanten Stakeholder ermittelt, analysiert und dokumentiert (Nuseibeh & Easterbrook, 2000, S. 37). Die Spezifikation konkretisiert diese Anforderungen durch Geschäftsobjekte, Geschäftsprozesse, Geschäftsregeln und Schnittstellen. Diese klassische Grundlage wird bereits in der Konzeptionsphase durch agile Elemente ergänzt. Dazu gehört insbesondere die Beschreibung funktionaler Anforderungen durch User Stories. Diese stellen Anforderungen aus Sicht einer bestimmten Rolle kompakt dar und werden durch Akzeptanzkriterien konkretisiert (Lucassen et al., 2016, S. 383–384). Bei Mystic Maze werden daher alle User Stories um überprüfbare Akzeptanzkriterien ergänzt.

In der Erarbeitungs- und Reflexionsphase werden zunächst die Softwarearchitektur ausgearbeitet und anschließend einzelne Funktionen schrittweise implementiert, getestet und bei Bedarf angepasst. Erkenntnisse aus Implementierung und Tests können dadurch zu einer Verfeinerung bestehender Anforderungen, Spezifikationen oder Architekturentscheidungen führen. Iterationen sind ein wesentlicher Bestandteil der agilen Softwareentwicklung und können zur Verbesserung der Softwarequalität beitragen, verursachen bei unkontrolliertem Einsatz jedoch auch zusätzlichen Zeit- und Arbeitsaufwand (Mumtaz et al., 2022, S. 65355–65356).

In der Finalisierungsphase wird überprüft, ob die entwickelte Anwendung die zuvor definierten Anforderungen und Akzeptanzkriterien erfüllt. Hierzu werden die durchgeführten Tests ausgewertet und die relevanten Teststufen betrachtet. Unit-Tests prüfen einzelne Komponenten bzw. Funktionen isoliert, während Integrationstests das Zusammenspiel mehrerer Komponenten überprüfen. Systemtests betrachten das Verhalten der vollständigen Anwendung als Gesamtsystem (Di Lucca & Fasolino, 2006, S.1176) . Die konkrete Durchführung und die Ergebnisse dieser Teststufen werden dokumentiert, um nachvollziehbar darzustellen, inwieweit die festgelegten Anforderungen erfüllt wurden. Anschließend wird das Vorgehen, aufgetretene Abweichungen, Limitationen und die Zielerreichung kritisch reflektiert. Die dabei gewonnenen Erkenntnisse werden als Lessons Learned festgehalten, um Erfahrungen aus dem Projekt für zukünftige Entwicklungsprojekte nutzbar zu machen (Abdellatif et al., 2019, S.44).

4. Projektübersicht

4.1 Projektziel

Ziel des Projekts ist die Konzeption und Entwicklung einer vollständig spielbaren Webanwendung, die das beschriebene Labyrinthspiel bereitstellt. Im Mittelpunkt stehen folgende Ziele:

- Entwicklung eines dynamisch veränderbaren Labyrinths auf einem rasterbasierten Spielfeld
- Umsetzung eines vollständigen, regelbasierten Spielablaufs
- Unterstützung von mindestens zwei Spieler an verschiedenen Endgeräten
- Kommunikation und Synchronisation der Spielzustände zwischen mindestens zwei Endgeräten innerhalb desselben lokalen Netzwerks beziehungsweise WLANs
- Integration von normalen und speziellen Labyrinthkacheln
- Integration verschiedener Zielkarten
- Verwendung unterschiedlicher Spielcharaktere mit individuellen Fähigkeiten
- grafische und intuitive Bedienung über Web-Graphic User Interface (GUI)
- saubere Trennung von Spielregeln, Spiellogik und GUI
- Entwicklung einer Anwendung, die automatisiert und manuell getestet werden kann

4.2 Spielprinzip

Zu Beginn einer Partie wird ein Labyrinth aus quadratischen Kacheln erzeugt. Die Kacheln besitzen unterschiedliche Wegverläufe:

- gerade Wege
- Kurven
- T-Kreuzungen
- Kreuzungen

Zusätzlich existieren besondere Kacheln mit zusätzlichen Spieleffekten.

Zu Beginn eines Spielzugs erhält der aktive Spieler eine freie Labyrinthkachel. Diese kann gedreht und anschließend an einer zulässigen Position in das Spielfeld eingeschoben werden. Dadurch verschiebt sich eine vollständige Reihe oder Spalte des Labyrinths und die gegenüberliegende Kachel wird aus dem Spielfeld herausgeschoben.

Anschließend darf sich die Spielfigur des aktiven Spielers über alle miteinander verbundenen Wege bewegen. Ziel ist es, bestimmte auf Zielkarten Artefakte zu erreichen.

Sobald das aktuelle Ziel erreicht wurde, wird die nächste Zielkarte aktiviert. Eine Partie endet, sobald ein Spieler alle ihm zugewiesenen Ziele erreicht hat.

Mystic Maze erweitert dieses grundlegende Spielprinzip insbesondere durch Spezialkacheln und individuelle Charakterfähigkeiten. Dadurch entstehen zusätzliche taktische Entscheidungsmöglichkeiten:

- Spezialkacheln

    - Portal: ermöglicht den Wechsel zu einem zweiten Portal
    - Tor: blockiert einen Weg, bis ein entsprechendes Schlüssel-Artefakt gefunden wurde
- Charakterfähigkeiten
    - Magier: darf einmal je gesuchtem Artefakt eine Kachel auf dem Spielfeld drehen
    - Wächter: darf Tore durchschreiten, ohne das benötigte Schlüssel-Artefakt zu besitzen

4.3 Zielgruppe

Die primäre Zielgruppe besteht aus Personen, die strategische Brett-, Puzzle- und Casual-Games bevorzugen. Angesprochen werden insbesondere:

- Jugendliche und Erwachsene
- Gelegenheitsspieler
- Fans taktischer Gesellschaftsspiele
- Personen, die lokale Mehrspieler-Spiele spielen möchten

Für das Spiel sind keine besonderen technischen Kenntnisse erforderlich.

4.4 Kundennutzen

Der Customer Value von Mystic Maze entsteht durch die Kombination aus leicht verständlicher Grundmechanik und taktischen Entscheidungsmöglichkeiten. Das Spiel kann im Webbrowser gestartet werden und benötigt kein physisches Spielmaterial. Die Software automatisiert dabei einen Teil der Spielorganisation. Ein weiterer Mehrwert entsteht dadurch, dass das Spiel von verschiedenen Personen am eigenen Endgerät gespielt werden kann. Dadurch entfällt die Notwendigkeit, sich einen Bildschirm oder Eingabegeräte zu teilen. Gleichzeitig ist für eine lokale Partie keine Bereitstellung eines öffentlich erreichbaren Internetservers erforderlich.

- automatische Prüfung der Einhaltung der Spielregeln
- automatische Ermittlung erreichbarer Felder
- automatisches Verschieben der Spielkacheln
- automatische Verwaltung von Zielkarten
- automatische Prüfung von Siegbedingungen
- keine Vorbereitung oder Sortierung von Spielmaterial erforderlich
- gemeinsames Spielen über mehrere Endgeräte

4.5 Projektumfang

Für eine kontrollierbare Umsetzung wird zwischen Muss-Funktionen und möglichen Erweiterungen unterschieden.

Geplanter Kernumfang:

- webbasierte GUI
- Spielbrett mit 7 × 7 Kacheln
- unterschiedliche Labyrinthkacheltypen
- Drehung der freien Kachel

- Einschieben von Reihen und Spalten
- automatische Aktualisierung des Spielfelds
- Spielfiguren
- Wegfindung und Prüfung erreichbarer Felder
- Einsammeln von Artefakten
- Zielkarten
- Artefakt-Kacheln
- Portal-Kacheln
- Tor-Kacheln
- zwei unterschiedliche Spielcharaktere mit Sonderfähigkeiten
- Prüfung der Siegbedingung
- Start einer neuen Partie
- zwei Spieler an verschiedenen Endgeräten
- Betrieb der Endgeräte innerhalb desselben lokalen Netzwerks beziehungsweise WLANs
- Erstellen beziehungsweise Hosten einer lokalen Partie
- Beitritt zu einer bestehenden lokalen Partie
- webbasierte Client-Server-Kommunikation
- zentrale Verwaltung des Spielzustands durch den Server
- Synchronisation von Spielaktionen und Spielzuständen zwischen den Endgeräten
- Behandlung eines Verbindungsabbruchs durch eine verständliche Fehlermeldung

Mögliche Erweiterungen:

- Speichern des Spielstandes
- Bis 4 Spieler an verschiedenen Endgeräten
- Computergegner
- weitere Charaktere mit Sonderfertigkeiten
- weitere Spezialkacheln
- Zufallsereignisse
- verschiedene Schwierigkeitsgrade
- Statistiken
- Highscore
- Soundeffekte und Hintergrundmusik

Diese Erweiterungen sind nachrangig. Im Folgenden wird nur die Umsetzung des Kernumfangs betrachtet.

5. Risikomanagement

Softwareprojekte unterliegen technischen, organisatorischen und zeitlichen Risiken. Für Mystic Maze werden die wichtigsten Risiken bereits während der Konzeptionsphase identifiziert und bewertet.

Bewertungsmethode

Für jedes Risiko werden zwei Faktoren auf einer Skala von 1 bis 5 bewertet:

- Eintrittswahrscheinlichkeit (E): 1 = sehr gering, 5 = sehr hoch
- Auswirkung (A): 1 = sehr gering, 5 = sehr schwerwiegend

Aus beiden Werten (E x A) ergibt sich das Risiko

Tabelle 1: Risikomatrix

ID	Risiko	E	A	Risiko	Gegenmaßnahme
R01	Projektumfang wird zu groß	4	5	20	MVP früh festlegen, Erweiterungen klar vom Kernumfang trennen
R02	Spielfeldlogik ist komplexer als erwartet	3	5	15	Spielfeld- und Verschiebelogik früh entwickeln und testen
R03	Fehler bei der Wegfindung	3	4	12	Algorithmus unabhängig von der GUI implementieren und Unit-Tests erstellen
R04	Charakterfähigkeiten führen zu Regelkonflikten	3	3	9	Fähigkeiten begrenzen und Geschäftsregeln eindeutig dokumentieren
R05	GUI benötigt mehr Entwicklungszeit als geplant	3	4	12	zunächst funktionale Minimal-GUI entwickeln, grafische Details später
R06	Projektverzögerung durch unerwartete Fehler	3	4	12	Zeitpuffer vor der Finalisierung einplanen
R07	Unzureichende Testabdeckung	3	4	12	Tests parallel zur Entwicklung statt ausschließlich am Projektende; Einsatz von Unittests während der Entwicklung

Quelle: Eigene Darstellung

Als größtes Risiko wird derzeit ein zu großer Projektumfang mit Risiko 20 bewertet. Die Vielzahl möglicher Funktionen kann dazu führen, dass zu viel Entwicklungszeit für optionale Funktionen aufgewendet wird. Daher wird von Beginn an ein klar definierter Minimalumfang festgelegt.

Ein weiteres wesentliches Risiko liegt in der Spielfeld- und Wegfindungslogik. Da sich das Labyrinth nach jedem Zug verändert, muss zuverlässig bestimmt werden können, welche Kacheln miteinander verbunden sind und welche Felder von einer Spielfigur erreicht werden können. Diese Funktion gehört deshalb zu den ersten zu implementierenden und zu testenden Komponenten.

6. Zeitplanung

Die Entwicklung wird in mehrere Arbeitspakete (AP) gegliedert. Dadurch können Projektfortschritt und mögliche Verzögerungen leichter überwacht werden.

Tabelle 2: Arbeitspakete

AP	**Arbeitspaket**	**Inhalt**
AP01	Projektkonzeption	Spielidee, Zielgruppe, Projektumfang
AP02	Projektdokumentation	Projektübersicht, Risiken, Zeitplanung
AP03	Anforderungsanalyse	Stakeholder, User Stories, Qualitätsanforderungen
AP04	Spezifikation	Datenmodell, Prozesse, Regeln und Schnittstellen
AP05	Architektur	Technologieauswahl und Softwarearchitektur
AP06	Implementierungs-setup	Entwicklungsumgebung einrichten
AP07	Grundlegende Spiello-gik	Spielfeld, Kacheln, Spieler, Spielzustand
AP08	Labyrinthmechanik	Drehen und Verschieben von Kacheln
AP09	Bewegungslogik	Ermittlung erreichbarer Kacheln
AP10	Ziele und Spezialme-chaniken	Zielkarten, Portale, Tore
AP11	Charaktere	Fähigkeiten und Regelprüfung
AP12	GUI	Spielfeld, Bedienung und Anzeigen
AP13	Server-Client-Kommu-nikation	Server-Client-Kommunikation, Verbindungsaufbau und Synchro-nisation des Spielzustands
AP14	Dokumentation Archi-tektur	Dokumentation der verwendeten Architektur
AP15	Qualitätssicherung	Unit-, Integrations- und Systemtests
AP16	Optimierung	Fehlerbehebung und Benutzerfreundlichkeit
AP17	Projektabschluss	Dokumentation, Benutzeranleitung und finale Abgabe

Quelle: Eigene Darstellung

Für einen Projektzeitraum von sechs Wochen sieht die Planung folgendermaßen aus:

Tabelle 3: Zeitplan

Woche	**Schwerpunkt**	**AP**
1	Projektplanung	AP01, AP02, AP03, AP04
2	Architekturauswahl, Projektsetup & Implementierungsstart	AP05, AP06, AP07, AP08
3	Implementierung	AP09, AP10, AP11
4	Implementierung & Dokumentation	AP12, AP13, AP14
5	Test & Optimierung	AP15, AP16
6	Projektabschluss	AP17

Quelle: Eigene Darstellung

Aus den Arbeitspaketen und der Zeitplanung lassen sich folgende Meilensteine (M) ableiten:

Tabelle 4: Meilensteine

M	AP	Beschreibung
M1: Projektplanung	01–04	Mit diesem Meilenstein sind die grundlegende Projektidee, die Projektplanung, die Anforderungen sowie die fachliche Spezifikation abgeschlossen. Damit liegt eine belastbare Grundlage für die anschließende technische Ausarbeitung und Umsetzung vor.
M2: Architekturauswahl	05	Die grundlegenden technologischen und architektonischen Entscheidungen für die Anwendung sind getroffen und dokumentiert. Dadurch ist festgelegt, auf welcher technischen Basis die weitere Entwicklung erfolgt.
M3: Grundspiel funktionsfähig	06–13	Zu diesem Zeitpunkt sind die wesentlichen Spielfunktionen umgesetzt. Dazu gehören insbesondere Spielfeld, Kachelmechanik, Bewegungslogik, Ziele, Spezialmechaniken, Charaktere und grafische Benutzeroberfläche, sodass eine vollständige Partie grundsätzlich durchgeführt werden kann.
M4: Dokumentation, Tests & Optimierung	14–16	Die verwendete Architektur wurde dokumentiert, die implementierte Anwendung wurde systematisch getestet und erkannte Fehler wurden behoben. Zusätzlich wurden Benutzerfreundlichkeit, Stabilität und das allgemeine Verhalten der Anwendung optimiert.
M5: Projektabschluss	17	Mit diesem Meilenstein sind die Anwendung und die zugehörigen Projektdokumente vollständig fertiggestellt. Die finale Version ist damit für die abschließende Abgabe vorbereitet.

Quelle: Eigene Darstellung

Literaturverzeichnis

Abdellatif, T. M., Capretz, L. F., & Ho, D. (2019). Automatic recall of software lessons learned for software project managers. *Information and Software Technology*, *115*, 44–57. https://doi.org/10.1016/j.infsof.2019.07.006

Di Lucca, G. A., & Fasolino, A. R. (2006). Testing Web-based applications: The state of the art and future trends. *Information and Software Technology*, *48*(12), 1172–1186. https://doi.org/10.1016/j.infsof.2006.06.006

Kuhrmann, M., Diebold, P., Münch, J., Tell, P., Garousi, V., Felderer, M., Trektere, K., McCaffery, F., Linssen, O., Hanser, E., & Prause, C. R. (2017). Hybrid software and system development in practice: waterfall, scrum, and beyond. *Proceedings of the 2017 International Conference on Software and System Process*, *10*, 30–39. https://doi.org/10.1145/3084100.3084104

Lucassen, G., Dalpiaz, F., van der Werf, J. M. E. M., & Brinkkemper, S. (2016). Improving agile requirements: the Quality User Story framework and tool. *Requirements Engineering*, *21*(3), 383–403. https://doi.org/10.1007/s00766-016-0250-x

Mishra, A., & Alzoubi, Y. I. (2023). Structured software development versus agile software development: a comparative analysis. *International Journal of System Assurance Engineering and Management*, *14*(4), 1504–1522. https://doi.org/10.1007/s13198-023-01958-5

Mumtaz, M., Ahmad, N., Usman Ashraf, M., Alghamdi, A. M., Bahaddad, A. A., & Almarhabi, K. A. (2022). Iteration Causes, Impact, and Timing in Software Development Lifecycle: An SLR. *IEEE Access*, *10*, 65355–65375. https://doi.org/10.1109/ACCESS.2022.3182703

Nuseibeh, B., & Easterbrook, S. (2000). Requirements engineering. *Proceedings of the Conference on The Future of Software Engineering*, 35–46. https://doi.org/10.1145/336512.336523

Ravensburger AG. (o. J.). *Das verrückte Labyrinth | Gesellschaftsspiele | Ravensburger*. Abgerufen 26. September 2026, von https://www.ravensburger.de/de-DE/produkte/spiele/gesellschaftsspiele-brettspiele/das-verrueckte-labyrinth-26955

Projekt: Software Engineering

(DLMCSPSE01_D)

IU Internationale Hochschule

Studiengang: Informatik

Anforderungsdokument

Jessica Krautheuser

Matrikelnummer: IU14129800

Tutor: Prof. Dr. Dirk Simon

Abgabedatum: 30.10.2026

Inhalt

Abbildungsverzeichnis III

Tabellenverzeichnis IV

Abkürzungsverzeichnis V

1. Einleitung 1

2. Stakeholder 1

2.1 Auftraggeber 1

2.2 Spieler 2

2.3 Entwickler 2

3. Funktionale Anforderungen 3

3.1 Spielstart und Verbindungsaufbau 3

3.2 Spielfeld 4

3.3 Bewegung der Spielfiguren 4

3.4 Zielkarten und Artefakte 5

3.5 Spezialkachel Portal und Tor 5

3.6 Charakterfähigkeiten 6

3.7 Zugsteuerung 7

3.8 Synchronisation und Client-Server-Kommunikation 7

3.9 Siegbedingung und Spielende 8

3.10 UML-Use-Case-Diagramm 8

4. Nichtfunktionale Anforderungen 9

5. Glossar 10

Abbildungsverzeichnis

Abbildung 1: UML-Use-Case-Diagramm 8

Tabellenverzeichnis

Tabelle 1: FA-01-05 3
Tabelle 2: FA-06-10 4
Tabelle 3: FA-11-13 4
Tabelle 4: FA-14-16 5
Tabelle 5: FA-17-20 5
Tabelle 6: FA-21-22 6
Tabelle 7: FA-23-25 7
Tabelle 8: FA-26-28 7
Tabelle 9: FA-29-30 8
Tabelle 10: NFA-01-14 9
Tabelle 11: Glossar 10

Abkürzungsverzeichnis

FA	Funktionale Anforderung
NFA	Nicht funktionale Anforderung
UML	Unified Modeling Language

1. Einleitung

Dieses Dokument beschreibt die Anforderungen an die Webanwendung Mystic Maze. Ziel ist es, die fachlichen und qualitativen Erwartungen an das Spiel eindeutig festzuhalten und damit eine verbindliche Grundlage für die spätere Spezifikation, Architektur, Implementierung und Qualitätssicherung zu schaffen.

Mystic Maze ist ein strategisches Labyrinthspiel für mindestens zwei Spieler. Die Spieler verwenden unterschiedliche Endgeräte innerhalb desselben lokalen Netzwerks beziehungsweise WLANs. Das Spielfeld besteht aus verschiebbaren Labyrinthkacheln. Durch deren gezielte Veränderung versuchen die Spieler, ihre individuellen Ziele zu erreichen und gleichzeitig die Möglichkeiten der gegnerischen Spielfigur zu beeinflussen.

2. Stakeholder

Unter Stakeholdern werden Personen oder Personengruppen verstanden, die die Anwendung verwenden, Anforderungen an sie stellen oder vom Projektergebnis betroffen sind.

2.1 Auftraggeber

Der Auftraggeber des Projekts stellt einen Stakeholder dar. Er definiert die übergeordneten Ziele und Erwartungen an die Anwendung und hat ein besonderes Interesse daran, dass die vereinbarten Anforderungen innerhalb des vorgesehenen Projektumfangs umgesetzt werden.

Interessen und Erwartungen:

- Erfüllung der festgelegten Projektziele
- Einhaltung des definierten Funktionsumfangs
- nachvollziehbare und strukturierte Projektumsetzung sowie Dokumentation
- funktionsfähige und qualitativ angemessene Anwendung
- Einhaltung des vorgesehenen Zeitrahmens

2.2 Spieler

Die Spieler stellen die wichtigste Stakeholdergruppe dar. Sie verwenden die Anwendung unmittelbar und erwarten einen verständlichen, stabilen und unterhaltsamen Spielablauf.

Zur primären Zielgruppe gehören insbesondere Jugendliche und Erwachsene, Gelegenheitsspieler sowie Personen mit Interesse an Brett-, Puzzle- und Casual-Games. Besondere technische Kenntnisse sollen zur Nutzung des Spiels nicht erforderlich sein.

Interessen und Erwartungen:

- intuitive Bedienung
- leicht verständliche Spielregeln
- übersichtliche Darstellung des Spielfelds
- nachvollziehbare Rückmeldung zu gültigen und ungültigen Aktionen
- zuverlässige Kommunikation zwischen den beteiligten Endgeräten
- identischer Spielzustand auf allen Endgeräten
- abwechslungsreiche taktische Möglichkeiten
- stabile Anwendung ohne unerwartete Abbrüche

2.3 Entwickler

Die entwickelnde Person ist ebenfalls Stakeholder des Projekts. Für die Entwicklung sind insbesondere eindeutig definierte Anforderungen, nachvollziehbare Geschäftsregeln und eine möglichst testbare Spiellogik relevant.

Interessen und Erwartungen:

- klar abgegrenzter Projektumfang
- eindeutige Anforderungen
- geringe Mehrdeutigkeit bei Spielregeln
- testbare Kernfunktionen
- wartbare und erweiterbare Anwendung

3. Funktionale Anforderungen

Die funktionalen Anforderungen (FA) beschreiben, welche Funktionen die Anwendung bereitstellen muss. Sie werden im Folgenden als User Stories formuliert.

Zur besseren Nachverfolgbarkeit erhält jede Anforderung eine eindeutige ID.

3.1 Spielstart und Verbindungsaufbau

Tabelle 1: FA-01-05

ID	User Story
FA-01	Als Spieler möchte ich eine neue Partie erstellen können, damit andere Spieler meiner Partie beitreten können.
FA-02	Als Spieler möchte ich einer bestehenden Partie über deren Partie-ID beitreten können, damit ich mit einer anderen Person auf einem separaten Endgerät spielen kann.
FA-03	Als Host-Spieler möchte ich die Partie starten können, sobald mindestens zwei Spieler:innen verbunden sind.
FA-04	Als Spieler möchte ich vor Spielbeginn einen verfügbaren Charakter auswählen können, damit ich dessen individuelle Fähigkeit während der Partie verwenden kann.
FA-05	Als Spieler möchte ich vor dem Erstellen oder Beitreten einer Partie einen Spielernamen festlegen können, damit ich innerhalb der Partie eindeutig dargestellt werden kann.

Quelle: Eigene Darstellung

Akzeptanzkriterien

- Eine neue Partie kann über einen Webbrowser erstellt werden
- Mindestens ein zweites Endgerät im selben Netzwerk kann der Partie beitreten
- Die Anwendung zeigt an, ob eine Verbindung besteht
- Der Spielbeginn ist erst möglich, wenn mindestens zwei Spieler verbunden sind
- Derselbe Charakter darf innerhalb einer Partie nicht mehrfach ausgewählt werden
- Nach dem Start erhalten alle Teilnehmer denselben initialen Spielzustand
- Jeder Spieler erhält einen Spielernamen zur eindeutigen Identifikation innerhalb des Spiels

3.2 Spielfeld

Tabelle 2: FA-06-10

ID	User Story
FA-06	Als Spieler möchte ich ein vollständig dargestelltes Labyrinth sehen, damit ich meine Spielzüge planen kann.
FA-07	Als Spieler möchte ich erkennen können, welche Kachel außerhalb des Spielfelds aktuell zum Einschieben zur Verfügung steht.
FA-08	Als Spieler möchte ich die freie Kachel vor dem Einschieben drehen können, damit ich den Verlauf des Labyrinths beeinflussen kann.
FA-09	Als Spieler möchte ich die freie Kachel an einer zulässigen Position in das Labyrinth einschieben können.
FA-10	Als Spieler möchte ich nach einer Verschiebung das aktualisierte Spielfeld sehen, damit der neue Spielzustand eindeutig erkennbar ist.

Quelle: Eigene Darstellung

Akzeptanzkriterien

- Das Spielfeld enthält 7 × 7 Kacheln.
- Die freie Kachel wird getrennt vom Spielfeld dargestellt und kann in 90° Schritten gedreht werden.
- Nur zulässige Einschubpositionen können ausgewählt werden.
- Beim Einschieben wird eine vollständige Reihe beziehungsweise Spalte verschoben.
- Die am gegenüberliegenden Rand herausgeschobene Kachel wird zur neuen freien Kachel.
- Der neue Zustand wird auf allen verbundenen Endgeräten synchron dargestellt.

3.3 Bewegung der Spielfiguren

Tabelle 3: FA-11-13

ID	User Story
FA-11	Als Spieler möchte ich nach dem Verschieben erkennen können, welche Felder meine Spielfigur erreichen kann.
FA-12	Als Spieler möchte ich meine Spielfigur auf ein erreichbares Feld bewegen können.
FA-13	Als Spieler möchte ich daran gehindert werden, meine Spielfigur auf ein nicht erreichbares Feld zu bewegen.

Quelle: Eigene Darstellung

Akzeptanzkriterien

- Nach Abschluss der Labyrinthverschiebung werden die vom aktuellen Standort erreichbaren Felder berechnet und für alle Spieler erkennbar dargestellt.
- Nur über miteinander verbundene Wege erreichbare Felder gelten als gültige Ziele.
- Eine Bewegung auf ein ungültiges Feld wird nicht durchgeführt.
- Nach einer gültigen Bewegung wird die neue Position auf allen Endgeräten dargestellt.

3.4 Zielkarten und Artefakte

Tabelle 4: FA-14-16

ID	User Story
FA-14	Als Spieler möchte ich mein aktuelles Ziel sehen, damit ich meine Spielzüge entsprechend planen kann.
FA-15	Als Spieler möchte ich ein Artefakt einsammeln, wenn ich dessen Position erreiche.
FA-16	Als Spieler möchte ich nach Erfüllung eines Ziels automatisch mein nächstes Ziel erhalten.

Quelle: Eigene Darstellung

Akzeptanzkriterien

- Jeder Spieler besitzt eigene Zielkarten.
- Nur das jeweils aktuelle Ziel wird als zu erfüllendes Ziel gewertet.
- Erreicht die Spielfigur das entsprechende Ziel, wird dieses als erfüllt markiert und das nächste Ziel aktiviert.

3.5 Spezialkachel Portal und Tor

Tabelle 5: FA-17-20

ID	User Story
FA-17	Als Spieler möchte ich, dass meine Figur beim Betreten einer Portalkachel auf der zweiten Portalkachel erscheint, damit ich zusätzliche Bewegungsmöglichkeiten erhalte.
FA-18	Als Spieler möchte ich erkennen können, welche beiden Portale miteinander verbunden sind.
FA-19	Als Spieler möchte ich ein verschlossenes Tor erst durchqueren können, wenn das benötigte Schlüssel-Artefakt aktiviert wurde.
FA-20	Als Spieler möchte ich erkennen können, dass ein Weg durch ein verschlossenes Tor momentan nicht passierbar ist.

Quelle: Eigene Darstellung

Akzeptanzkriterien

- zwei zusammengehörige Portal-Kacheln befinden sich im Spiel.
- Befindet sich eine Spielfigur auf einem Portal, erscheint sie auf der zweiten Portalkachel
- Die Portalnutzung darf nicht zu einem ungültigen Spielzustand führen.
- Die neue Position wird auf allen Endgeräten synchronisiert.
- Ein geschlossenes Tor wird bei der Berechnung erreichbarer Felder als nicht passierbar behandelt.
- Wurde das Schlüsselobjekt aktiviert, darf ein Spieler das Tor passieren.
- Die grafische Darstellung macht den Zustand des Tors erkennbar.

3.6 Charakterfähigkeiten

Tabelle 6: FA-21-22

ID	User Story
FA-21	Als Spieler mit dem Charakter Magier möchte ich einmal für jedes gesuchte Artefakt eine zusätzliche Kachel drehen können, damit ich das Labyrinth zu meinem Vorteil verändern kann.
FA-22	Als Spieler mit dem Charakter Wächter möchte ich Tore ohne das zugehörige Schlüssel-Artefakt durchschreiten können, damit mein Charakter einen besonderen taktischen Vorteil besitzt.

Quelle: Eigene Darstellung

Akzeptanzkriterien

- Die Fähigkeit gilt nur für den entsprechenden Charakter
- Die Fähigkeit des Charakters Magier kann nur einmal pro gesuchtem Artefakt aktiviert werden und wird danach als verbraucht angezeigt.
- Bei Fähigkeit des Wächters werden bei der Berechnung seiner erreichbaren Felder geschlossene Tore als passierbar behandelt. Andere Charaktere benötigen weiterhin das aktivierte Schlüssel-Artefakt um zu passieren.

3.7 Zugsteuerung

Tabelle 7: FA-23-25

ID	User Story
FA-23	Als Spieler möchte ich erkennen können, wann ich am Zug bin.
FA-24	Als Spieler möchte ich meinen Zug nach Abschluss meiner Aktionen beenden können.
FA-25	Als Spieler möchte ich keine spielverändernden Aktionen durchführen können, wenn ein anderer Spieler am Zug ist.

Quelle: Eigene Darstellung

Akzeptanzkriterien

- Der aktive Spieler wird eindeutig angezeigt.
- Nur der aktive Spieler kann spielverändernde Aktionen ausführen.
- Nach Zugende wechselt die aktive Spielerrolle zum nächsten Spieler.
- Der Spielerwechsel wird auf allen Endgeräten synchronisiert.

3.8 Synchronisation und Client-Server-Kommunikation

Tabelle 8: FA-26-28

ID	User Story
FA-26	Als Spieler möchte ich in meinem Webbrowser denselben Spielzustand sehen wie meine Mitspieler, damit eine gemeinsame Partie möglich ist.
FA-27	Als Spieler möchte ich, dass meine gültigen Spielaktionen automatisch bei allen verbundenen Spielern aktualisiert werden, damit alle den aktuellen gemeinsamen Spielzustand sehen
FA-28	Als Spieler möchte ich über einen Verlust der Verbindung zum Webserver informiert werden, damit ich weiß, warum die Partie nicht fortgesetzt werden kann.

Quelle: Eigene Darstellung

Akzeptanzkriterien

- Nach einer gültigen Aktion müssen bei allen verbundenen Spielern dieselben Informationen vorliegen.
- Bei einem Verbindungsfehler wird eine verständliche Meldung angezeigt und die Anwendung darf nicht unkontrolliert beendet werden.

3.9 Siegbedingung und Spielende

Tabelle 9: FA-29-30

ID	User Story
FA-29	Als Spieler möchte ich gewinnen können, wenn ich alle meiner Zielkarten erfüllt habe.
FA-30	Als Spieler möchte ich nach Spielende eindeutig erkennen können, wer die Partie gewonnen hat.

Quelle: Eigene Darstellung

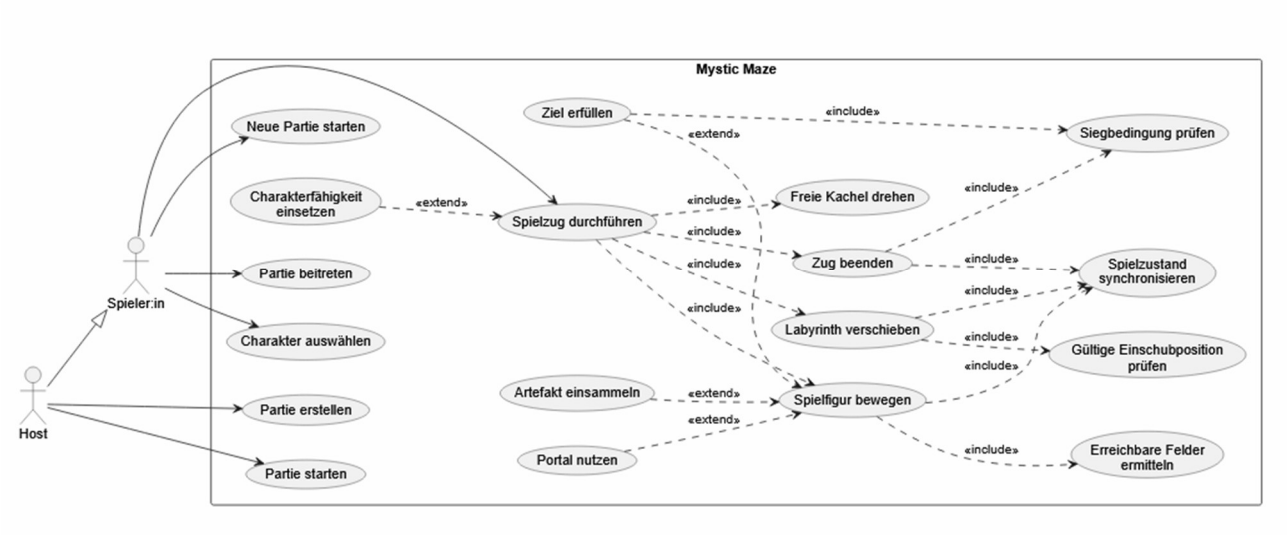
Akzeptanzkriterien

- Die Siegbedingung wird nach Erfüllung eines Ziels oder am Ende eines Zuges geprüft.
- Eine Partie endet nur, wenn ein Spieler alle ihm zugewiesenen Ziele erfüllt hat
- Der Gewinner wird für alle verbundenen Spieler eindeutig angezeigt.
- Der finale Spielzustand wird auf allen verbundenen Browser-Sitzungen konsistent dargestellt.

3.10 UML-Use-Case-Diagramm

Ein Unified Modelling Language (UML)-Use-Case-Diagramm stellt die funktionalen Beziehungen zwischen den Akteuren eines Systems und den von ihnen genutzten Anwendungsfällen dar. Es dient dazu, die wesentlichen Interaktionen mit der Anwendung übersichtlich zu visualisieren, ohne dabei technische Implementierungsdetails abzubilden. Das folgende UML-Use-Case-Diagramm visualisiert die zentralen Interaktionen zwischen den Spieler und der Anwendung. Es zeigt die wesentlichen Funktionen des Spielstarts, des Spielablaufs sowie des Spielendes und verdeutlicht die Beziehungen zwischen den einzelnen Anwendungsfällen.

Abbildung 1: UML-Use-Case-Diagramm



Quelle: Eigene Darstellung

4. Nichtfunktionale Anforderungen

Nichtfunktionale Anforderungen (NFA) beschreiben die Qualitätsmerkmale der Anwendung. Für Mystic Maze sind insbesondere Benutzerfreundlichkeit, Performance, Zuverlässigkeit, Konsistenz, Wartbarkeit und Skalierbarkeit relevant.

Tabelle 10: NFA-01-14

ID	Kategorie	Anforderung
NFA-01	Benutzer-freundlichkeit	Die Anwendung muss ohne besondere technische Kenntnisse bedienbar sein.
NFA-02	Benutzer-freundlichkeit	Aktuell mögliche beziehungsweise relevante Aktionen müssen für Spieler eindeutig erkennbar sein.
NFA-03	Benutzer-freundlichkeit	Ungültige Aktionen müssen verhindert oder durch eine verständliche Rückmeldung erläutert werden.
NFA-04	Benutzer-freundlichkeit	Der aktive Spieler, das aktuelle Ziel und der Verbindungsstatus müssen eindeutig dargestellt werden.
NFA-05	Performance	Lokale Spielaktionen sollen ohne für den Benutzer störende Verzögerungen dargestellt werden.
NFA-06	Konsistenz	Nach einer erfolgreich synchronisierten Aktion müssen alle verbundenen Endgeräte denselben für die Partie relevanten Spielzustand besitzen.
NFA-07	Zuverlässigkeit	Ungültige Benutzereingaben oder Spielaktionen dürfen nicht zu einem Absturz der Anwendung führen.
NFA-08	Zuverlässigkeit	Ein Verlust der Netzwerkverbindung darf nicht zu einem unkontrollierten Programmabbruch führen.
NFA-09	Wartbarkeit	Spiellogik und grafische Benutzeroberfläche sollen so getrennt sein, dass zentrale Spielregeln unabhängig von der GUI getestet werden können.
NFA-10	Verständlichkeit	Fehlermeldungen müssen für Benutzer verständlich formuliert sein und dürfen keine rein technischen Fehlermeldungen darstellen.
NFA-11	Sicherheit	Über das Netzwerk empfangene Spielaktionen dürfen nur verarbeitet werden, wenn sie dem erwarteten Nachrichtenformat entsprechen.
NFA-12	Sicherheit	Ein Client darf keine Spielaktion erfolgreich ausführen können, wenn er nach den Spielregeln nicht an der Reihe ist.
NFA-13	Kompatibilität	Die Webanwendung muss mit den gängigen Webbrowsern, wie Chrome, Edge und Firefox kompatibel sein.

NFA-14	Skalierbarkeit	Die Webanwendung soll so ausgelegt sein, dass die unterstützte Spielerzahl pro Partie von zwei auf bis zu vier Spieler erweitert werden kann, ohne die zentrale Spiellogik grundlegend anzupassen.

Quelle: Eigene Darstellung

5. Glossar

Das folgende Glossar erläutert die im Dokument verwendeten projektspezifischen Begriffe, um ein einheitliches Verständnis der Anforderungen und Spielmechaniken sicherzustellen.

Tabelle 11: Glossar

Begriff	Beschreibung
Artefakt	Sammelbarer Gegenstand auf dem Spielfeld, der als Ziel oder Schlüssel dienen kann.
Charakter	Vom Spieler gewählte Spielfigur mit einer individuellen Fähigkeit.
Webanwendung	Anwendung, deren Benutzeroberfläche über einen Webbrowser aufgerufen wird und deren zentrale Spiellogik beziehungsweise Partieverwaltung serverseitig bereitgestellt wird.
Endgerät	Computer, über das ein Spieler einen Webbrowser aufruft um Mystic Maze auszuführen.
Freie Kachel	Labyrinthkachel, die sich momentan außerhalb des 7×7-Spielfelds befindet und im nächsten Zug eingeschoben werden kann.
GUI	*Graphical User Interface*; grafische Benutzeroberfläche der Anwendung.
Host	Spieler, der innerhalb der Webanwendung eine neue Partie erstellt und den Spielstart steuert.
Kachel	Quadratisches Element des Spielfelds mit einem bestimmten Wegverlauf oder einer besonderen Funktion.
Kurve	Standardkachel mit zwei im rechten Winkel verbundenen Wegeöffnungen.
Labyrinth	Gesamtheit der auf dem 7×7-Spielfeld angeordneten Kacheln.
Magier	Spielcharakter mit der Fähigkeit, einmal pro gesuchtem Artefakt eine zusätzliche Kachel auf dem Spielplan zu drehen.
Portal	Spezialkachel, die eine zusätzliche Verbindung zu einer zugehörigen Portal-Kachel herstellt.
Spielzustand	Gesamtheit aller Daten, die den aktuellen Zustand einer Partie beschreiben, beispielsweise Kachelanordnung, Spielerpositionen, Ziele und aktiver Spieler.
Spezialkachel	Labyrinthkachel mit einer zusätzlichen Funktion über den normalen Wegverlauf hinaus.

T-Kreuzung	Standardkachel mit drei miteinander verbundenen Wegeöffnungen.
Tor	Spezialkachel mit einem zunächst blockierten Weg, der normalerweise nur mit dem benötigten Schlüssel-Artefakt durchquert werden darf.
Wächter	Spielcharakter, der Tore ohne das benötigte Schlüssel-Artefakt passieren kann.
Wegfindung	Ermittlung aller Felder, die eine Spielfigur anhand der aktuellen Verbindungen des Labyrinths erreichen kann.
Webbrowser	Programm zur Darstellung und Bedienung der Webanwendung, beispielsweise Microsoft Edge, Mozilla Firefox oder Google Chrome.
Webserver	Serverseitiger Bestandteil der Webanwendung, der Partien und verbindliche Spielzustände verwaltet, Spielaktionen validiert und Änderungen an verbundene Browserclients verteilt.
Zielkarte	Vorgabe für ein vom Spieler zu erreichendes Artefakt.
Zug	Gesamtheit der Aktionen, die ein Spieler während seiner Spielrunde durchführen darf.

Quelle: Eigene Darstellung



Projekt: Software Engineering

(DLMCSPSE01_D)

IU Internationale Hochschule

Studiengang: Informatik

Spezifikationsdokument

Jessica Krautheuser

Matrikelnummer: IU14129800

Tutor: Prof. Dr. Dirk Simon

Abgabedatum: 30.10.2026

Inhalt

Abbildungsverzeichnis III

Tabellenverzeichnis IV

Abkürzungsverzeichnis V

1. Einleitung 1

2. Datenmodell 1

3. Geschäftsprozesse 3

3.1 Partie erstellen und beitreten 3

3.2 Partie durchführen 4

4. Geschäftsregeln 7

5. Systemkontext und Systemschnittstellen 9

5.1 Systemkontext 9

5.2 Systemschnittstellen 10

6. Benutzerschnittstellen 11

Abbildungsverzeichnis

Abbildung 1: UML-Klassendiagramm 2
Abbildung 2: UML-Aktivitätsdiagramm Partie erstellen und beitreten 4
Abbildung 3: UML-Aktivitätsdiagramm Partie durchführen 6
Abbildung 4: Kontextdiagramm 9
Abbildung 5: Mockup der Startseite 11
Abbildung 6: Mockup Spielvorbereitung 12
Abbildung 7: Mockup Spielansicht 13
Abbildung 8: Mockup Spielende 14
Abbildung 9: Dialogfluss 15

Tabellenverzeichnis

Tabelle 1: Geschäftsobjekte 1
Tabelle 2: Geschäftsregeln 7
Tabelle 3: Übertragene Daten und Aktionen 10
Tabelle 4: Eingabevalidierungen 15

Abkürzungsverzeichnis

GO	Geschäftsobjekt
GR	Geschäftsregel
GUI	Graphic User Interface
JSON	JavaScript Object Notation
UML	Unified Modeling Language

1. Einleitung

Dieses Dokument spezifiziert die fachlich und technisch nach außen sichtbaren Eigenschaften der Webanwendung Mystic Maze. Beschrieben werden das fachliche Datenmodell, die zentralen Geschäftsprozesse und Geschäftsregeln, der Systemkontext und Systemschnittstellen sowie die Benutzerschnittstellen.

2. Datenmodell

Das fachliche Datenmodell beschreibt die zentralen Geschäftsobjekte (GO) der Webanwendung und ihre Beziehung zueinander. Es legt fest, welche Informationen für die Durchführung einer Partie fachlich benötigt werden, ohne technische Implementierungsdetails vorwegzunehmen.

Tabelle 1: Geschäftsobjekte

GO	**Zweck**	**Zentrale Informationen**
Partie	Repräsentiert eine vorbereitete, laufende oder beendete Spielrunde.	Partie-ID, Status, teilnehmende Spieler, aktiver Spieler, Gewinner
Spieler	Repräsentiert eine teilnehmende Person innerhalb einer Partie.	Spieler-ID, Name, Position
Charakter	Vom Spieler gewählter Charakter mit individueller Sonderfähigkeit.	Name, Sonderfähigkeit, Nutzungsstatus
Spielfeld	Enthält das 7 × 7-Kachelraster sowie die aktuell freie Kachel. Von den 49 Kacheln auf dem Spielfeld sind 16 fest.	Größe, freie Kachel
Kachel	Ein einzelnes Element des Labyrinths mit Wegverlauf und Orientierung.	Position, Orientierung, Wegtyp
Zielkarte	Beschreibt das aktuell zu erreichende Artefakt.	Ziel-ID, Typ, Erfüllungsstatus
Artefakt	Sammelbarer Gegenstand, der als Ziel oder Schlüssel dienen kann. Insgesamt gibt es 24 Artefakte	Artefakt-ID, Bezeichnung
Spielzug	Fasst die fachlichen Aktionen eines Spielers innerhalb einer Runde zusammen.	Zugnummer, Zugphase

Quelle: Eigene Darstellung

Die Partie bildet den übergeordneten Rahmen des Datenmodells. Ihr sind die Spieler und genau ein Spielfeld zugeordnet. Der Status der Partie beschreibt sowohl die Vorbereitungsphase als auch die laufende und abgeschlossene Partie.

Jeder Spieler wählt für die Partie einen Charakter. Zusätzlich können einem Spieler mehrere Zielkarten und eingesammelte Artefakte zugeordnet sein.

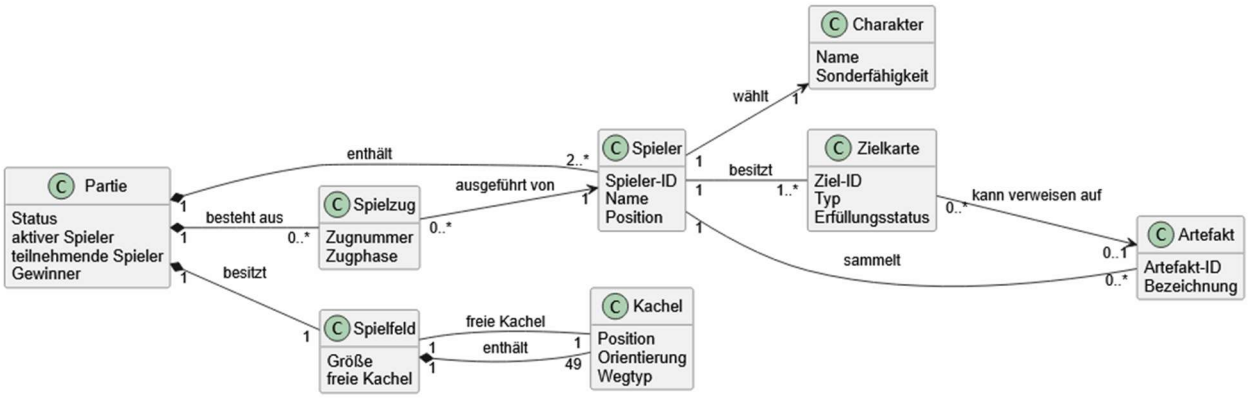
Das Spielfeld enthält 49 innerhalb des Rasters angeordnete Kacheln sowie eine zusätzliche freie Kachel. Spezialkacheln stellen besondere Ausprägungen einer Kachel dar.

Eine Zielkarte gehört genau einem Spieler und verweist auf ein Artefakt.

Eine Partie besteht im Zeitverlauf aus mehreren Spielzügen. Jeder Spielzug wird von genau einem Spieler ausgeführt.

Im folgenden UML-Klassendiagramm werden die zentralen Geschäftsobjekte und ihre Beziehungen visualisiert dargestellt.

Abbildung 1: UML-Klassendiagramm



Quelle: Eigene Darstellung

3. Geschäftsprozesse

Die Geschäftsprozesse beschreiben die zentralen fachlichen Abläufe der Anwendung aus Sicht der Spieler und des Systems. Für Mystic Maze werden die Prozesse „Partie erstellen und beitreten“ sowie „Partie durchführen“ betrachtet. Der Prozess „Partie durchführen“ umfasst die wiederkehrenden Spielzüge bis zum Eintritt der Siegbedingung.

3.1 Partie erstellen und beitreten

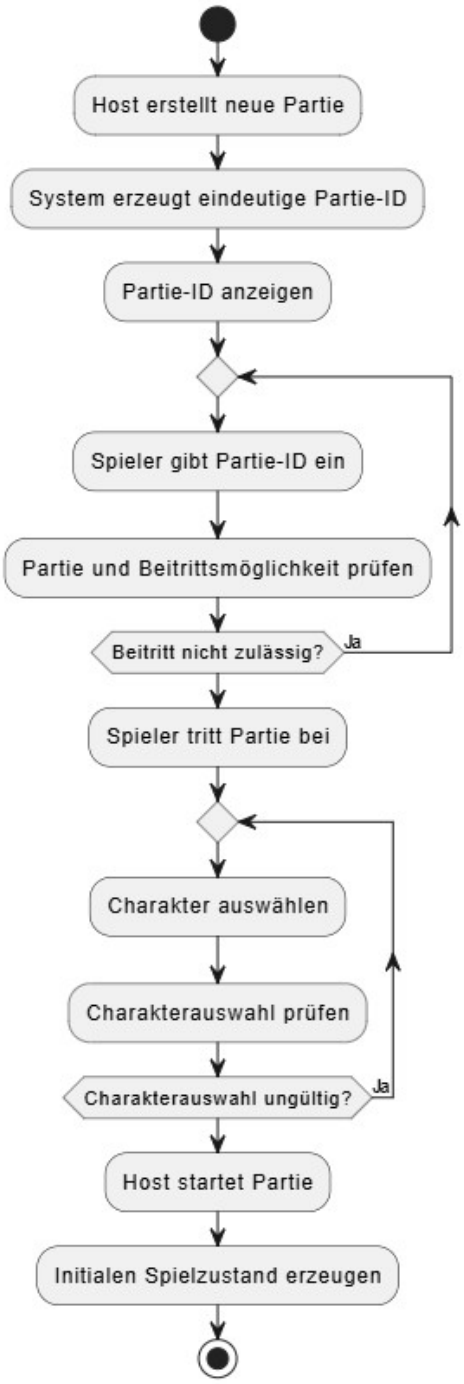
Vor Beginn einer Partie müssen die beteiligten Spieler einer gemeinsamen Partie zugeordnet werden und jeweils einen verfügbaren Charakter auswählen.

Der Ablauf gestaltet sich wie folgt:

1. Der Host erstellt über die Webanwendung eine neue Partie.
2. Das System erzeugt für die Partie eine eindeutige Partie-ID und zeigt diese dem Host an.
3. Ein weiterer Spieler ruft die Webanwendung auf und gibt die Partie-ID ein.
4. Das System prüft, ob die angegebene Partie existiert und ein Beitritt zulässig ist.
5. Ist ein Beitritt nicht möglich, wird der Spieler darüber informiert und kann eine andere Partie-ID eingeben.
6. Bei erfolgreicher Prüfung tritt der Spieler der Partie bei.
7. Die Spieler wählen jeweils einen verfügbaren Charakter.
8. Das System prüft, ob der ausgewählte Charakter innerhalb der Partie noch verfügbar ist. Bereits vergebene Charaktere können nicht erneut gewählt werden.
9. Sobald mindestens zwei Spieler beigetreten sind und die erforderlichen Voraussetzungen erfüllt sind, kann der Host die Partie starten.
10. Das System erzeugt den initialen Spielzustand und die Partie beginnt.

Der Prozess wird im folgenden erstellten UML-Aktivitätsdiagramm dargestellt:

Abbildung 2: UML-Aktivitätsdiagramm Partie erstellen und beitreten



Quelle: Eigene Darstellung

3.2 Partie durchführen

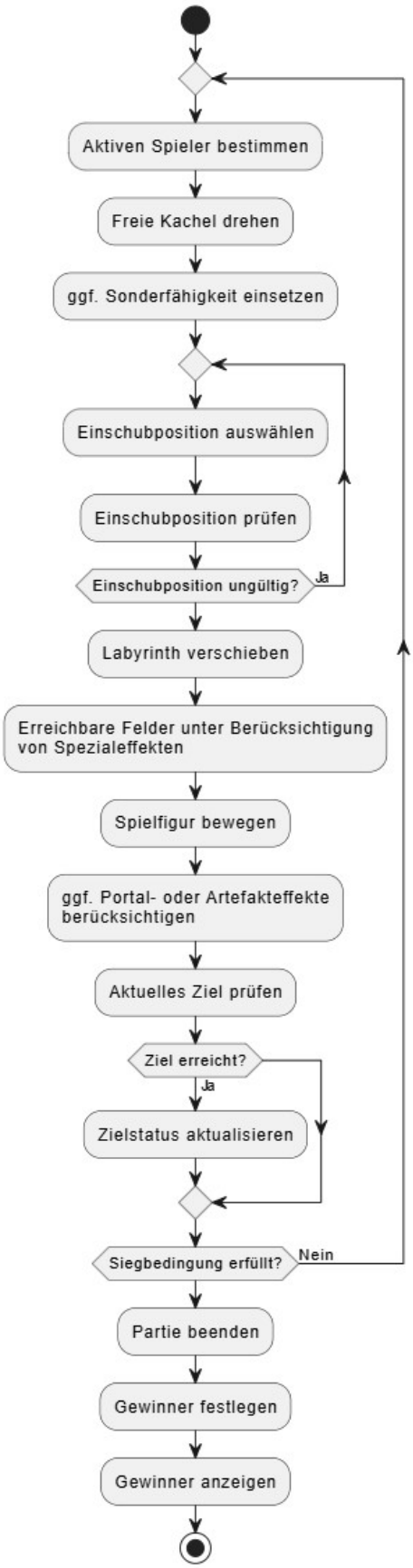
Nach dem Start der Partie führen die Spieler abwechselnd Spielzüge durch. Innerhalb eines Spielzugs wird zunächst das Labyrinth verändert und anschließend die Spielfigur bewegt. Charakterfähigkeiten, Spezialkacheln, Artefakte und Zielkarten können den normalen Ablauf beeinflussen.

Der Ablauf einer Partie gestaltet sich wie folgt:

1. Das System bestimmt den aktiven Spieler. Nur dieser darf spielverändernde Aktionen durchführen.
2. Der aktive Spieler kann die freie Kachel drehen.
3. Sofern für den gewählten Charakter eine aktive Sonderfähigkeit zur Verfügung steht und deren Voraussetzungen erfüllt sind, kann diese eingesetzt werden.
4. Der Spieler wählt eine Einschubposition für die freie Kachel.
5. Das System prüft, ob die ausgewählte Einschubposition zulässig ist. Bei einer ungültigen Auswahl muss eine andere Position gewählt werden.
6. Die freie Kachel wird eingeschoben und die entsprechende Reihe oder Spalte des Labyrinths verschoben. Die gegenüberliegende Kachel wird dabei zur neuen freien Kachel.
7. Das System ermittelt die für die aktive Spielfigur erreichbaren Felder. Dabei werden relevante Spezialeffekte wie Tore, Schlüssel-Artefakte und Charakterfähigkeiten berücksichtigt.
8. Der Spieler bewegt seine Spielfigur auf ein erreichbares Feld.
9. Gegebenenfalls werden durch die neue Position Portal- oder Artefakteffekte ausgelöst.
10. Das System prüft, ob das aktuell aktive Ziel erreicht wurde.
11. Wurde das Ziel erreicht, wird dessen Status aktualisiert und, sofern weitere Ziele vorhanden sind, das nächste Ziel aktiviert.
12. Anschließend wird geprüft, ob der Spieler alle ihm zugewiesenen Ziele erfüllt hat.
13. Ist die Siegbedingung nicht erfüllt, beendet der aktive Spieler seinen Spielzug. Anschließend beginnt mit dem nächsten aktiven Spieler ein weiterer Spielzug.
14. Ist die Siegbedingung erfüllt, wird die Partie beendet und der Gewinner festgelegt und angezeigt. Weitere spielverändernde Aktionen sind anschließend nicht mehr möglich.

Der Prozess „Partie durchführen“ wird im folgenden UML-Aktivitätsdiagramm dargestellt. Das Diagramm zeigt den wiederkehrenden Ablauf der einzelnen Spielzüge bis zur Erfüllung der Siegbedingung.

Abbildung 3: UML-Aktivitätsdiagramm Partie durchführen



4. Geschäftsregeln

Die folgenden Geschäftsregeln (GR) sind während einer Partie verbindlich einzuhalten und konkretisieren das zulässige Verhalten der zentralen Geschäftsobjekte und Prozesse.

Tabelle 2: Geschäftsregeln

ID	Geschäftsregel
GR-01	Eine Partie kann erst gestartet werden, wenn mindestens zwei Spieler beigetreten sind.
GR-02	Jeder Spieler besitzt innerhalb einer Partie genau einen Charakter.
GR-03	Ein bereits vergebener Charakter kann innerhalb derselben Partie nicht erneut gewählt werden.
GR-04	Nur der aktive Spieler darf spielverändernde Aktionen durchführen.
GR-05	Vor der Bewegung der Spielfigur muss die Labyrinthverschiebung des aktuellen Zuges abgeschlossen sein.
GR-06	Die freie Kachel darf nur an einer zulässigen Einschubposition in das Spielfeld eingeschoben werden.
GR-07	Beim Einschieben wird die gegenüberliegende Kachel aus dem Spielfeld herausgeschoben und zur neuen freien Kachel.
GR-08	Eine Spielfigur darf nur über miteinander verbundene Wege erreichbare Felder betreten.
GR-09	Jedes Tor ist genau einem Schlüssel-Artefakt zugeordnet. Sobald das Schlüssel-Artefakt von einem Spieler aktiviert wurde, darf das Tor passiert werden.
GR-10	Der Charakter Wächter darf geschlossene Tore ohne das zugehörige Schlüssel-Artefakt passieren.
GR-11	Die aktive Sonderfähigkeit des Magiers darf einmal pro gesuchten Artefakt eingesetzt werden.
GR-12	Ein Ziel gilt nur dann als erfüllt, wenn das aktuell aktive Ziel erreicht wurde.
GR-13	Nach Erfüllung eines Ziels wird die nächste Zielkarte des Spielers aktiviert, sofern eine weitere Zielkarte vorhanden ist.
GR-14	Der Spielzustand muss nach einer gültigen spielverändernden Aktion für alle verbundenen Spieler konsistent sein.

GR-15	Nach Eintritt der Siegbedingung dürfen keine weiteren spielverändernden Aktionen durchgeführt werden.
GR-16	Das Spielfeld besteht aus 49 Kacheln in einem 7 × 7-Raster und einer zusätzlichen freien Kachel.
GR-17	Innerhalb des Spielfelds sind 16 Kacheln fest positioniert und können durch eine Labyrinthverschiebung nicht ihre Position ändern.
GR-18	Eine Partie enthält insgesamt 24 Artefakte. Davon werden 12 Artefakte auf fest positionierten und zwölf auf beweglichen Kacheln positioniert. Dabei dürfen auf Start, Tor- und Portalkacheln kein Artefakt positioniert werden.

Quelle: Eigene Darstellung

5. Systemkontext und Systemschnittstellen

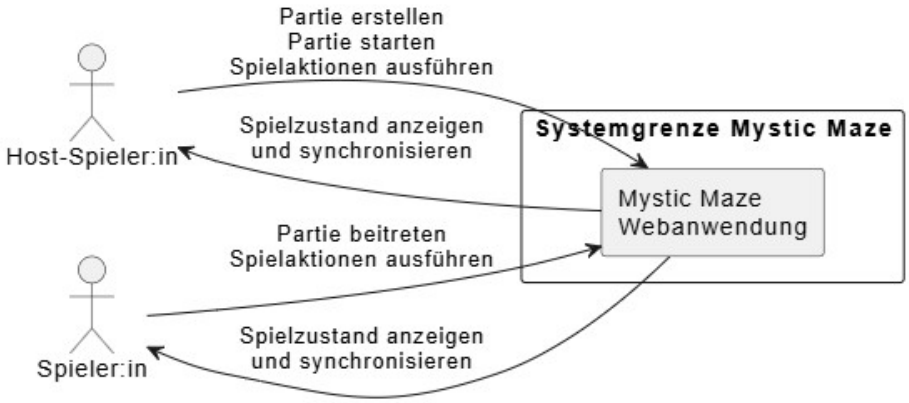
5.1 Systemkontext

Mystic Maze wird als Webanwendung umgesetzt und von mehreren Spielern über einen Webbrowser auf unterschiedlichen Endgeräten verwendet. Die Anwendung bildet die zentrale Systemgrenze und umfasst die Verwaltung von Partien, Spieler, Spielzuständen und Spielaktionen. Innerhalb des Systems werden unter anderem das Spielfeld, die Zugreihenfolge, die Zielkarten, Charakterfähigkeiten sowie die Prüfung der Spielregeln verwaltet.

Außerhalb der Systemgrenze befinden sich die Benutzer und ihre Endgeräte. Der Host-Spieler erstellt innerhalb der Anwendung eine neue Partie und startet diese, sobald die erforderliche Anzahl an Spieler verbunden ist. Weitere Spieler treten der Partie über die Webanwendung bei. Sämtliche spielrelevanten Aktionen werden über die Anwendung verarbeitet und der aktuelle Spielzustand anschließend für alle beteiligten Browser-Sitzungen synchronisiert.

Der Systemkontext umfasst damit insbesondere die Interaktion zwischen Spieler, Webbrowsern und der Webanwendung. Das folgende Kontextdiagramm stellt die Systemgrenze von Mystic Maze sowie die wesentlichen externen Akteure und deren Interaktionen mit der Anwendung dar.

Abbildung 4: Kontextdiagramm



Quelle: Eigene Darstellung

5.2 Systemschnittstellen

Mystic Maze besitzt keine Schnittstellen zu fachlichen Drittsystemen. Für die Nutzung als Webanwendung ist jedoch eine technische Kommunikationsschnittstelle zwischen den Webbrowsern der Spieler und dem Webserver erforderlich.

Für den Aufruf der Webanwendung und klassische Anfrage-Antwort-Kommunikation wird HTTP verwendet. Für die laufende Partie ist zusätzlich eine Kommunikation zwischen Browser und Webserver vorgesehen, um Änderungen des Spielzustands zeitnah an alle beteiligten Spieler zu übertragen. Hierfür wird eine WebSocket-Verbindung verwendet. Die zwischen Browser und Webserver ausgetauschten strukturierten Daten werden im JSON-Format (JavaScript Object Notation) übertragen. JSON ermöglicht eine kompakte und eindeutig strukturierte Darstellung von Spielaktionen und Spielzuständen.

Spielaktionen werden vom Browser an den Webserver übertragen. Der Server prüft, ob die Aktion im aktuellen Spielzustand zulässig ist. Gültige Änderungen werden in den zentralen Spielzustand übernommen und anschließend an die beteiligten Browser-Sitzungen verteilt. Ungültige Aktionen werden abgewiesen und dem auslösenden Benutzer verständlich zurückgemeldet.

Tabelle 3: Übertragene Daten und Aktionen

Aktion / Information	Richtung	Inhalt
Partie erstellen	Browser → Server	Anforderung zur Erzeugung einer neuen Partie
Partie beitreten	Browser → Server	Partie-ID und spielerbezogene Beitrittsinformationen.
Charakter auswählen	Browser → Server	Ausgewählter Charakter.
Kachel drehen	Browser → Server	Neue Orientierung der freien Kachel.
Labyrinth verschieben	Browser → Server	Gewählte Einschubposition und -richtung.
Spielfigur bewegen	Browser → Server	Gewähltes Zielfeld.
Sonderfähigkeit nutzen	Browser → Server	Aktivierung der verfügbaren Charakterfähigkeit.
Spielzustand	Server → Browser	Kachelanordnung, freie Kachel, Spielerpositionen, aktiver Spieler, Ziele, Artefakte, Fähigkeiten und Torzustände.
Fehler / Validierung	Server → Browser	Verständliche Rückmeldung zu ungültigen oder nicht ausführbaren Aktionen.

Quelle: Eigene Darstellung

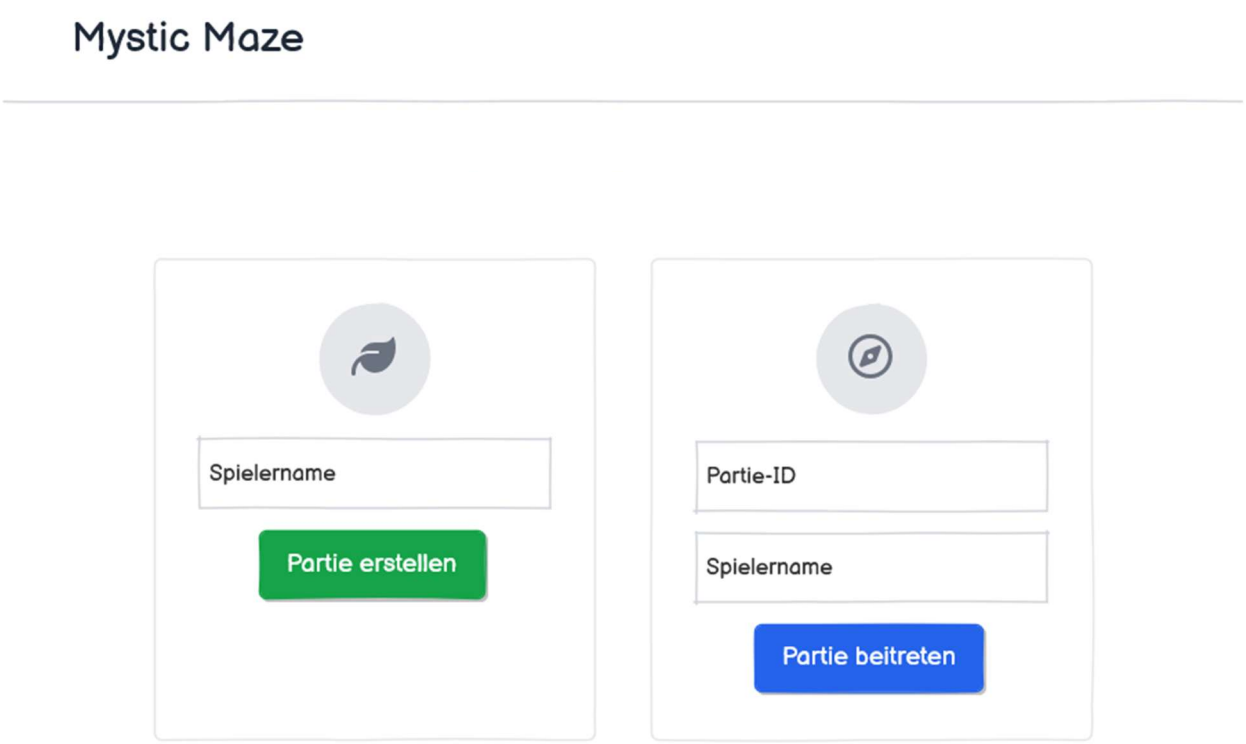
6. Benutzerschnittstellen

Die grafische Benutzeroberfläche wird über einen Webbrowser bereitgestellt. Die Bedienung soll ohne besondere technische Kenntnisse möglich sein. Die folgenden Ansichten bilden die zentralen Dialoge der Anwendung. Zur Veranschaulichung wurden Mockups der einzelnen Oberflächen erstellt. Sie stellen kein finales Design der GUI dar, sondern einen ersten Entwurf dieser.

Startseite:

- Schaltfläche zum Erstellen einer neuen Partie.
- Eingabefeld für eine bestehende Partie-ID.
- Schaltfläche zum Beitreten einer Partie.
- Verständliche Rückmeldung bei ungültiger oder nicht vorhandener Partie-ID.

Abbildung 5: Mockup der Startseite

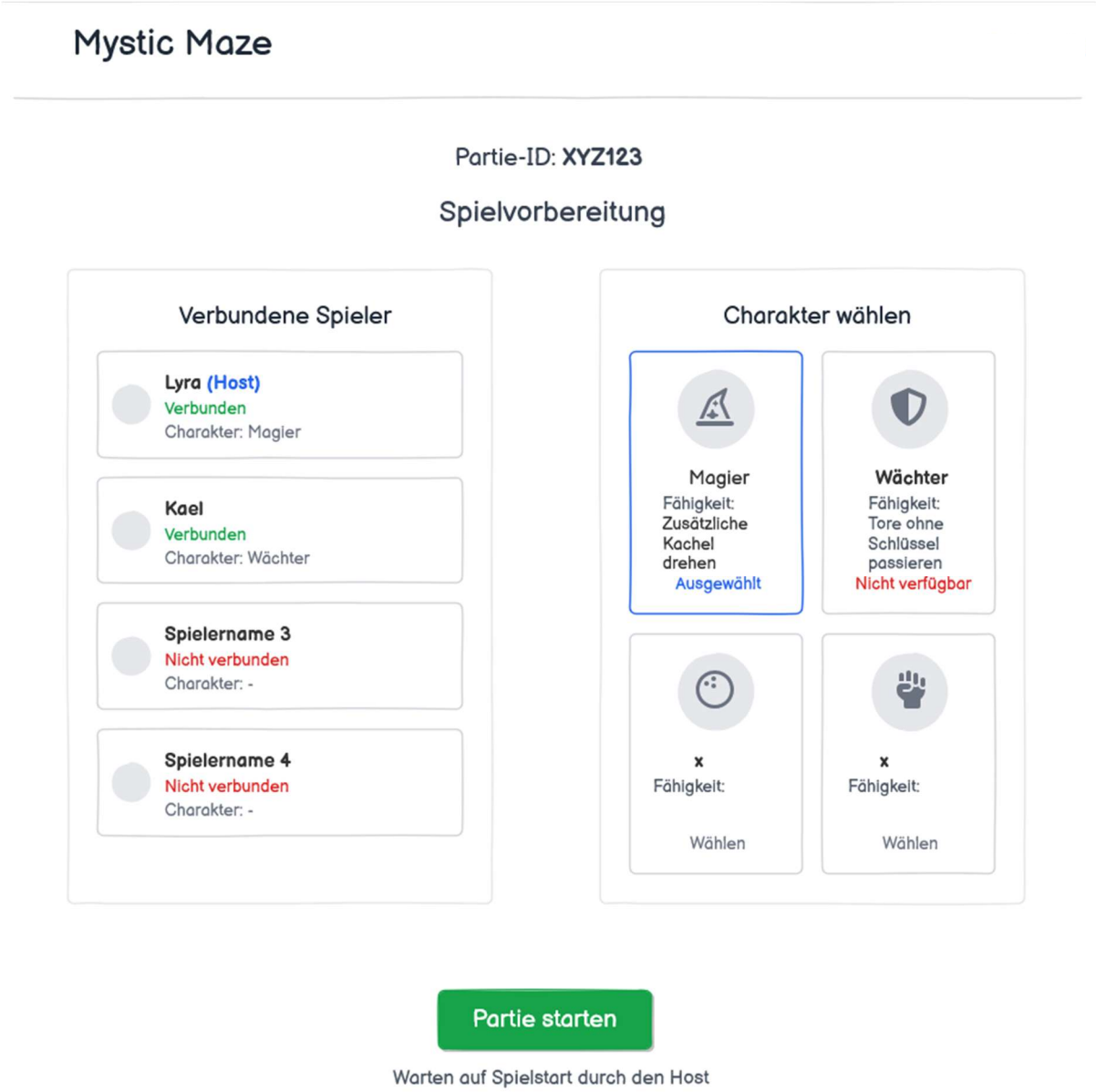


Quelle: Eigene Darstellung

Spielvorbereitung:

- Anzeige der Partie-ID.
- Anzeige der bereits beigetretenen Spieler.
- Auswahl eines verfügbaren Charakters.
- Kennzeichnung bereits vergebener Charaktere.
- Startmöglichkeit für den Host, sobald die erforderlichen Voraussetzungen erfüllt sind.

Abbildung 6: Mockup Spielvorbereitung

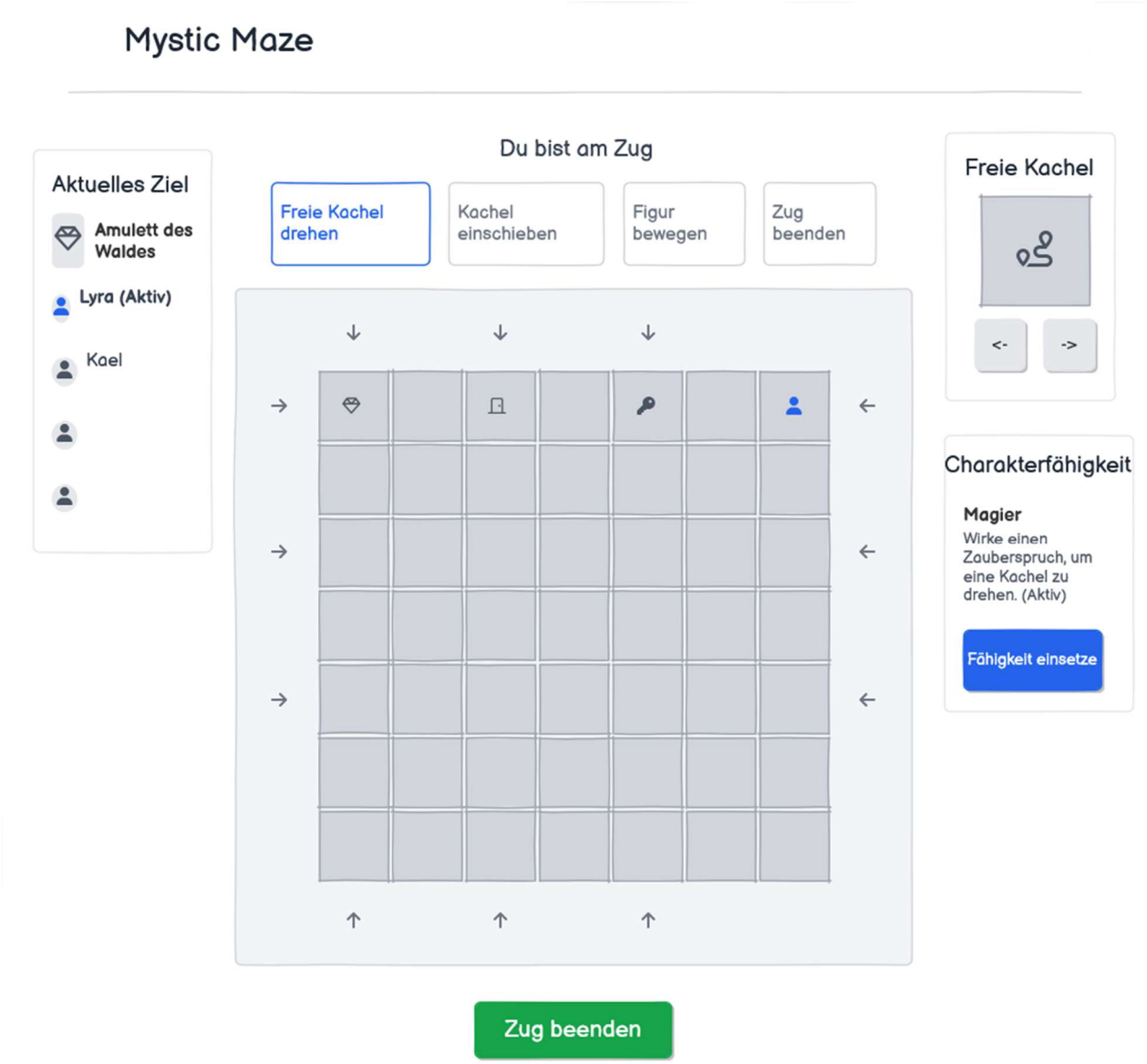


Quelle: Eigene Darstellung

Spielansicht:

- Darstellung des vollständigen 7 × 7-Spielfelds und der Spielfiguren.
- Getrennte Darstellung der freien Kachel mit Möglichkeit zur Drehung.
- Kennzeichnung zulässiger Einschubpositionen.
- Kennzeichnung erreichbarer Felder für die aktive Spielfigur.
- Anzeige des aktiven Spielers und des aktuellen Ziels.
- Anzeige des gewählten Charakters und des Status seiner Sonderfähigkeit.
- Darstellung von Portalen, Toren oder Artefakten.
- Schaltfläche zum Beenden des eigenen Zuges.
- Sichtbare Rückmeldung über den Verbindungsstatus.

Abbildung 7: Mockup Spielansicht

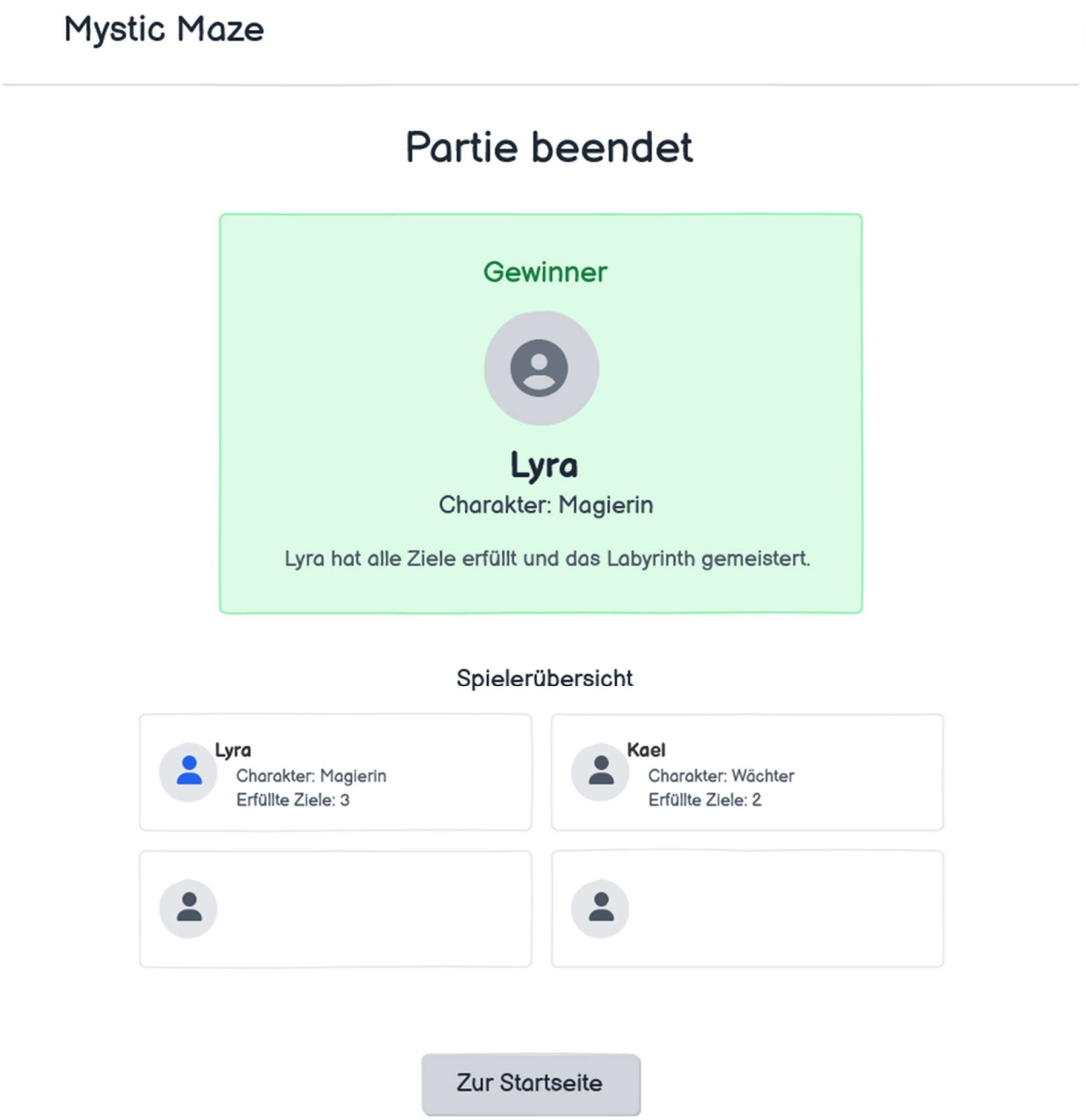


Quelle: Eigene Darstellung

Spielende:

- Eindeutige Anzeige des Gewinners.
- Keine weiteren spielverändernden Aktionen nach Ende der Partie.
- Möglichkeit zum Start einer neuen Partie.

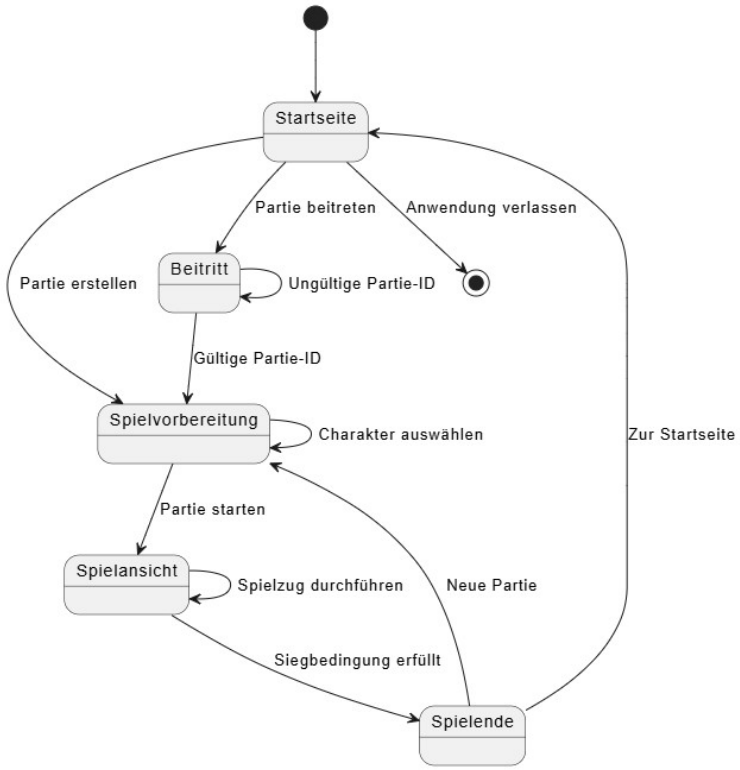
Abbildung 8: Mockup Spielende



Quelle: Eigene Darstellung

Das folgende Diagramm zeigt den grundlegenden Dialogfluss zwischen Startseite, Warteseite, Spielansicht und Spielende.

Abbildung 9: Dialogfluss



Quelle: Eigene Darstellung

In der folgenden Tabelle werden Validierungen für Benutzereingaben in der GUI festgehalten.

Tabelle 4: Eingabevalidierungen

Eingabe / Aktion	Validierung	Systemreaktion bei ungültiger Eingabe
Partie-ID	Die eingegebene Partie muss existieren und einen Beitritt erlauben.	Beitritt wird verhindert und eine verständliche Fehlermeldung angezeigt.
Charakterauswahl	Der Charakter muss verfügbar sein.	Bereits vergebene Charaktere sind nicht auswählbar.
Kacheleinschub	Die gewählte Position muss zulässig sein und der Spieler muss am Zug sein.	Aktion wird nicht ausgeführt; zulässige Positionen bleiben erkennbar.
Spielfigur bewegen	Das Zielfeld muss vom aktuellen Standort aus erreichbar sein.	Bewegung wird verhindert.
Sonderfähigkeit	Die Fähigkeit muss für den Charakter zulässig und ggf. noch verfügbar sein.	Aktivierung wird verhindert und der aktuelle Status angezeigt.
Zug beenden	Die Aktion muss vom aktiven Spieler ausgelöst werden.	Aktion wird abgewiesen.

Quelle: Eigene Darstellung

Projekt: Software Engineering

(DLMCSPSE01_D)

IU Internationale Hochschule

Studiengang: Informatik

Architekturdokument

Jessica Krautheuser

Matrikelnummer: IU14129800

Tutor: Prof. Dr. Dirk Simon

Abgabedatum: 30.10.2026

Inhalt

Abbildungsverzeichnis III

Abkürzungsverzeichnis IV

1. Einleitung 1

2. Technologieübersicht 1

2.1 Python 1

2.2 HTML, CSS und JavaScript 2

2.3 FastAPI, Uvicorn und Client-Server-Kommunikation 2

2.4 pytest 3

2.5 Entwicklungs- und Dokumentationswerkzeuge 4

3. Architekturübersicht 5

4. Struktur 7

5. Verhalten 11

Literaturverzeichnis IV

Abbildungsverzeichnis

Abbildung 1: Architekturdiagramm 6
Abbildung 2: UML-Klassendiagramm 10
Abbildung 3: Sequenzdiagramm Spielzug 12

Abkürzungsverzeichnis

ASGI	Asynchronous Server Gateway Interface
CSS	Cascading Style Sheets
GUI	Graphic User Interface
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrierte Entwicklungsumgebung
JSON	JavaScript Object Notation
PyPI	Python Package Index
TLS	Transport Layer Security
UML	Unified Modeling Language

1. Einleitung

Dieses Dokument beschreibt die Softwarearchitektur der Webanwendung Mystic Maze. Ziel ist es, die technischen Grundlagen für die Implementierung festzulegen und die interne Struktur der Anwendung nachvollziehbar zu dokumentieren. Dazu werden die eingesetzten Technologien und Werkzeuge, die gewählte Architektur sowie die zentralen Komponenten und ihre Beziehungen beschrieben. Ergänzend werden die Struktur der Anwendung durch ein UML-Klassendiagramm und ein zentraler Systemablauf durch ein UML-Sequenzdiagramm dargestellt. Das Architekturdokument baut auf den zuvor erstellten Anforderungen und der Spezifikation auf und dient als technische Grundlage für die weitere Implementierung der Anwendung.

2. Technologieübersicht

Für die Umsetzung von Mystic Maze werden Technologien eingesetzt, die eine browserbasierte Mehrspieler-Anwendung mit zentral verwalteter Spiellogik und synchronisierten Spielzuständen ermöglichen. Bei der Auswahl wird insbesondere auf eine klare Trennung der Spiellogik von der Benutzeroberfläche, eine einfache Client-Server-Kommunikation sowie eine gute Testbarkeit geachtet.

2.1 Python

Die serverseitige Anwendung und die Spiellogik werden in Python implementiert. Für den Einsatz der Programmiersprache spricht das bereits vorhandene Know-how bei der Entwicklerin. Dadurch ist nur eine geringe Einarbeitung notwendig und der Fokus kann auf der Umsetzung der Spiellogik liegen.

Python besitzt eine übersichtliche Syntax und unterstützt objektorientierte Programmierung. Dadurch lassen sich Bestandteile des Spiels wie Spielfeld, Kacheln, Spieler und Partien durch einzelne Klassen abbilden. Die offizielle Python-Dokumentation beschreibt Python als leicht zu erlernende Programmiersprache mit einer klaren Syntax sowie einer umfangreichen Standardbibliothek (Python Software Foundation, o. J.-b, o. S.).

Ein weiterer Vorteil ist die umfangreiche Auswahl von Bibliotheken, Modulen und Frameworks. Über den Python Package Index (PyPI) steht eine große Anzahl zusätzlicher Softwarepakete zur Verfügung, sodass benötigte Funktionen bei Bedarf durch etablierte Bibliotheken ergänzt werden können (Python Software Foundation, o. J.-a, o. S.). Für Mystic Maze kann Python deshalb sowohl für die Spiellogik als auch für die serverseitige Bereitstellung der Anwendung verwendet werden.

2.2 HTML, CSS und JavaScript

Die grafische Benutzeroberfläche und das Frontend von Mystic Maze wird mit HTML (HyperText Markup Language), CSS (Cascading Style Sheets) und JavaScript umgesetzt. HTML bildet die Struktur der einzelnen Webseiten und deren Inhalte. CSS wird für die Gestaltung und Anordnung der Oberflächenelemente verwendet. JavaScript übernimmt die Interaktivität und das dynamische Verhalten der Anwendung (Mozilla, o. J., o. S.).

Bei Mystic Maze verarbeitet JavaScript die Eingaben der Spieler und übernimmt die Kommunikation mit dem Server. Dazu gehören bspw. das Drehen und Einschieben einer Kachel, die Auswahl eines Zielfeldes und das Beenden eines Spielzugs. Außerdem werden die vom Server empfangenen Änderungen des Spielzustands durch JavaScript in der Benutzeroberfläche dargestellt.

Für die Kombination aus HTML, CSS und JavaScript spricht vor allem, dass alle drei Technologien direkt von modernen Webbrowsern unterstützt werden. Die Spieler benötigen deshalb keine zusätzliche Software, sondern können das Spiel über ihren Browser aufrufen. Außerdem lassen sich die Struktur, Gestaltung und Funktionen der Benutzeroberfläche getrennt voneinander entwickeln.

Auf ein zusätzliches Frontend-Framework wird verzichtet, da sich der benötigte Funktionsumfang mit diesen drei Technologien umsetzen lässt. Dadurch werden zusätzliche Abhängigkeiten und der damit verbundene Entwicklungsaufwand vermieden. Die eigentliche Spiellogik bleibt weiterhin auf dem Python-Server und wird nicht im Browser ausgeführt.

2.3 FastAPI, Uvicorn und Client-Server-Kommunikation

Als serverseitiges Webframework wird FastAPI eingesetzt. FastAPI ist ein Python-Framework zur Entwicklung von Web-APIs und basiert auf standardmäßigen Python-Typannotationen (FastAPI, o. J.-a, o. S.). Für die Definition und Validierung strukturierter Ein- und Ausgabedaten wird ergänzend Pydantic verwendet. Die Datenmodelle werden dabei mithilfe von Python-Typannotationen beschrieben und von FastAPI direkt für die Verarbeitung von Anfragen und Antworten genutzt.

Für einzelne Anfragen zwischen Browser und Server wird das Hypertext Transfer Protocol (HTTP) verwendet. HTTP ist ein zustandsloses Protokoll auf Anwendungsebene und basiert auf einem Request-Response-Prinzip zwischen Client und Server (Fielding et al., 2022, S. 19). Bei Mystic Maze wird HTTP beispielsweise für das Aufrufen der Webanwendung sowie für das Erstellen, Beitreten und Starten einer Partie verwendet. Statt HTTP wäre es auch möglich HTTPS zu verwenden. HTTPS ermöglicht durch den Einsatz von TLS (Transport Layer Security) eine verschlüsselte Kommunikation zwischen Webbrowser und Webserver (Fielding et al., 2022, S. 25). Da Mystic Maze ausschließlich für die Nutzung innerhalb eines lokalen WLANs vorgesehen ist, wird im Rahmen des Projektumfangs auf die zusätzliche Einrichtung und Verwaltung von TLS-Zertifikaten verzichtet. Die Datenübertragung erfolgt daher unverschlüsselt.

Während der Spielvorbereitung und einer laufenden Partie müssen Änderungen des Spielzustands auch ohne eine neue HTTP-Anfrage an die verbundenen Spieler übertragen werden können. Dafür wird das WebSocket-Protokoll verwendet. WebSocket ermöglicht nach dem Verbindungsaufbau eine bidirektionale Kommunikation zwischen Client und Server. Beide Seiten können dabei unabhängig voneinander Nachrichten übertragen (Fette & Melnikov, 2011, S. 5). Bei Mystic Maze werden über diese Verbindung Spielaktionen an den Server gesendet und Änderungen des Spielzustands an die beteiligten Clients verteilt.

Für die Übertragung strukturierter Daten zwischen Browser und Server wird JSON (JavaScript Object Notation) verwendet. JSON ist ein textbasiertes und sprachunabhängiges Format zur Darstellung strukturierter Daten (Bray, 2017, S. 1) . Dadurch können beispielsweise Partie-IDs, Spielerinformationen, Spielaktionen, Positionen und weitere Bestandteile des Spielzustands in einer einheitlichen Form zwischen dem JavaScript-Frontend und dem Python-Backend ausgetauscht werden.

Für Mystic Maze wurde FastAPI als Framework gewählt, da es sowohl HTTP-Endpunkte als auch WebSocket-Verbindungen unterstützt. Dadurch können beide Kommunikationsarten mit einem Framework umgesetzt werden. FastAPI stellt außerdem Funktionen zum Senden und Empfangen von Text-, Binär- und JSON-Daten über WebSocket-Verbindungen bereit (FastAPI, o. J.-c, o. S.).

Zur Ausführung der FastAPI-Anwendung wird Uvicorn verwendet. FastAPI basiert auf dem Asynchronous Server Gateway Interface (ASGI) und benötigt für die Bereitstellung der Anwendung einen entsprechenden ASGI-Server. Uvicorn übernimmt diese Aufgabe und verarbeitet die eingehenden HTTP- und WebSocket-Verbindungen (FastAPI, o. J.-b, o. S.). Dadurch kann die serverseitige Anwendung von Mystic Maze im lokalen Netzwerk bereitgestellt werden.

2.4 pytest

Für die automatisierten Tests von Mystic Maze wird das Python-Testframework pytest verwendet. Mit pytest lassen sich Tests erstellen und automatisch ausführen. Die Testbedingungen können dabei mit den normalen assert-Anweisungen von Python überprüft werden. Schlägt ein Test fehl, zeigt pytest zusätzliche Informationen zu den beteiligten Werten an und erleichtert dadurch die Fehlersuche. Außerdem erkennt pytest Testdateien und Testfunktionen anhand festgelegter Namenskonventionen automatisch (pytest.org, o. J., o. S.).

Bei Mystic Maze wird pytest vor allem zur Überprüfung der Spiellogik eingesetzt. Dazu gehören beispielsweise die Erstellung und Verschiebung des Spielfelds, die Ermittlung erreichbarer Positionen und die Einhaltung der Spielregeln. Da die Spiellogik von der Benutzeroberfläche getrennt ist, können diese Funktionen unabhängig vom Browser getestet werden. So lassen sich Fehler in den Spielmechaniken bereits während der Entwicklung erkennen.

Für die Wahl von pytest spricht außerdem, dass die Anwendung bereits in Python entwickelt wird. Die Tests können deshalb in derselben Programmiersprache geschrieben werden und benötigen

kein zusätzliches Testframework einer anderen Sprache. Bei Änderungen an der Spiellogik können die Tests erneut ausgeführt und auch für die abschließende Qualitätssicherung verwendet werden.

2.5 Entwicklungs- und Dokumentationswerkzeuge

Für die Entwicklung von Mystic Maze wird PyCharm als integrierte Entwicklungsumgebung (IDE) verwendet. PyCharm bietet Funktionen zur Bearbeitung und Verwaltung des Python-Codes sowie zur Ausführung und Fehlersuche während der Entwicklung (JetBrains, o. J.-b, o. S.). Für die Wahl von PyCharm spricht vor allem die Unterstützung der Programmiersprache Python und die Möglichkeit, den Quellcode, die Tests und die Ausführung der Anwendung innerhalb einer Entwicklungsumgebung zu verwalten.

Zusätzlich wird in PyCharm das Plugin AI Assistant mit dem KI-Agenten Codex von OpenAI verwendet. Codex kann auf den Projektcode zugreifen und bei der Entwicklung neuer Funktionen, der Fehlersuche und der Bearbeitung bestehender Dateien unterstützen (JetBrains, o. J.-a, o. S.). Bei Mystic Maze wird Codex für die Fehlerbehebung und die Gestaltung der Benutzeroberfläche eingesetzt.

Für die Erstellung der Grafiken von Mystic Maze wird der Chatbot ChatGPT von OpenAI verwendet. Dazu gehören beispielsweise die Labyrinthkacheln, der Waldbodenhintergrund und verschiedene Symbole für die Benutzeroberfläche. Die Grafiken werden mithilfe der integrierten Bildgenerierung durch Textanweisungen erstellt und anschließend durch weitere Anweisungen angepasst (OpenAI, o. J., o. S.).

Zur Versionsverwaltung wird Git eingesetzt. Git ist ein verteiltes Versionsverwaltungssystem, mit dem unterschiedliche Entwicklungsstände und Änderungen am Quellcode nachvollzogen werden können (Git, o. J., o. S.). Das zugehörige Repository wird auf GitHub bereitgestellt. Ein GitHub-Repository kann Quellcode und weitere Projektdateien einschließlich ihrer Versionshistorie zentral verwalten (GitHub, o. J., o. S.).

Zur Modellierung von UML-Diagrammen wird das Tool PlantUML eingesetzt. Mit PlantUML können Diagramme durch einen textbasierten Code beschrieben und daraus automatisch grafische Darstellungen erzeugt werden (PlantUML, o. J., o. S.). Dadurch lassen sich die Diagramme bei Änderungen an der Architektur direkt im Code anpassen, ohne die einzelnen Elemente manuell verschieben oder neu zeichnen zu müssen.

Die Mockups der Benutzeroberfläche im Spezifikationsdokument wurden mithilfe von Balsamiq erstellt. Balsamiq ist ein Werkzeug zur Erstellung von Mockups, mit denen die grundlegende Struktur und Anordnung von Elementen einer GUI bereits vor deren Implementierung visualisiert werden kann (Balsamiq, o. J., o. S.).

3. Architekturübersicht

Mystic Maze wird als webbasierte Client-Server-Anwendung umgesetzt. Bei einer Client-Server-Architektur fordert ein Client Dienste beziehungsweise Daten von einem Server an, während der Server diese bereitstellt und die entsprechenden Anfragen verarbeitet. Webanwendungen stellen einen typischen Anwendungsfall dieser Architektur dar (Salah et al., 2016, S. 319).

Bei Mystic Maze übernehmen die Webbrowser der Spieler die Rolle der Clients. Sie stellen die GUI dar, erfassen Benutzeraktionen und übertragen diese an den Server. Die zentrale Spiellogik und der verbindliche Spielzustand werden dagegen serverseitig verwaltet. Dadurch existiert für jede Partie ein eindeutiger Zustand, der nicht unabhängig auf den einzelnen Endgeräten berechnet werden muss.

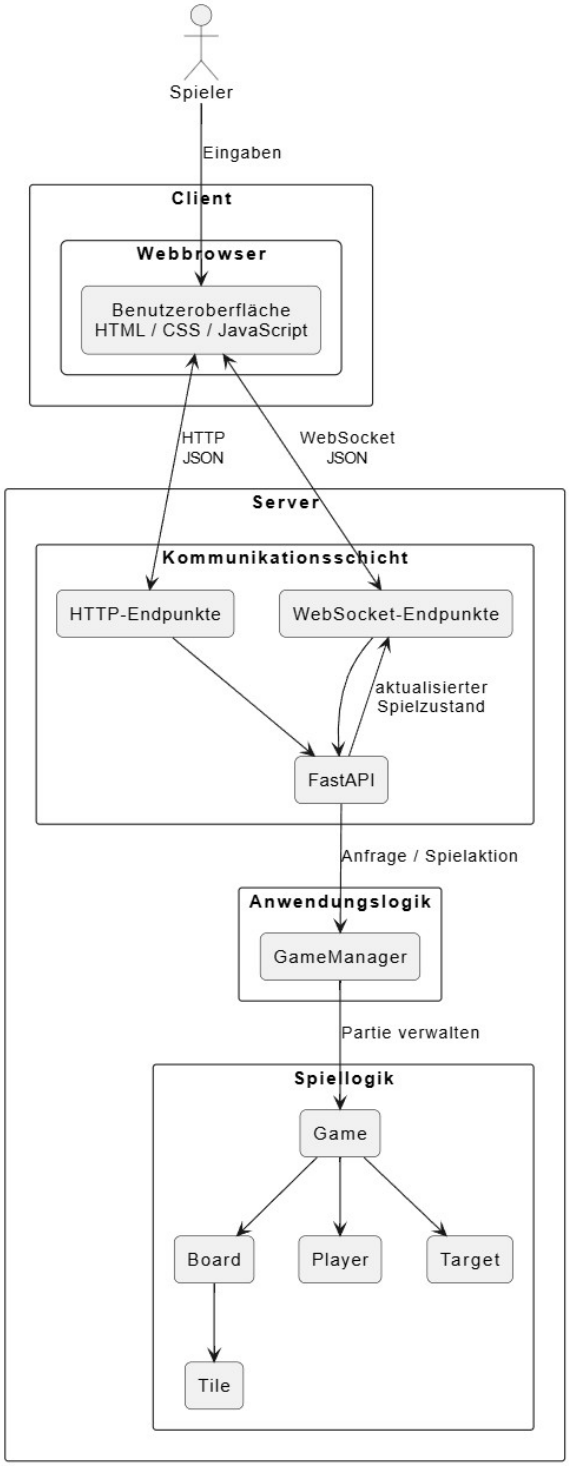
Bei Übertragung der Spielzustände wird zwischen öffentlichen und privaten Informationen unterschieden. Öffentliche Informationen sind Informationen, die für alle Spieler relevant sind, dazu zählen u.a. das Spielfeld, die Positionen der Spieler und der Zugstatus. Persönliche Informationen werden nur für einen Spieler gebraucht, bspw. das aktuell gesuchte Ziel des Spielers, und werden auch nur an den Client dieses Spielers gesendet.

Diese zentrale Verwaltung ist insbesondere für den Mehrspielermodus relevant. Aktionen eines Spielers, beispielsweise das Drehen oder Einschieben einer Kachel oder die Bewegung einer Spielfigur, werden zunächst an den Server übertragen. Dort wird geprüft, ob die Aktion im aktuellen Spielzustand zulässig ist. Nur gültige Aktionen verändern den zentralen Spielzustand. Anschließend wird der aktualisierte Zustand an die verbundenen Clients verteilt. Dadurch soll verhindert werden, dass unterschiedliche Endgeräte voneinander abweichende Spielzustände verwenden.

Innerhalb der serverseitigen Anwendung werden die Verantwortlichkeiten zusätzlich in voneinander abgegrenzte Bereiche unterteilt. Dieses Vorgehen orientiert sich am Prinzip einer geschichteten Architektur. In einem solchen System sind die einzelnen Schichten hierarchisch angeordnet und stellen jeweils Dienste für andere Schichten bereit. Durch die Aufteilung in einzelne Schichten können Änderungen an einem Bereich vorgenommen werden, ohne dass andere Bereiche direkt davon betroffen sind (Garlan & Shaw, 1993, S. 9-11).

Für Mystic Maze wird zwischen Benutzeroberfläche, Kommunikationsschicht, Anwendungslogik und Spiellogik unterschieden. Die daraus resultierende grundlegende Architektur ist in der folgenden Abbildung dargestellt. Die Abbildung zeigt die Aufteilung der Anwendung in den browserbasierten Client und die serverseitigen Bereiche sowie die wesentlichen Kommunikations- und Abhängigkeitsbeziehungen zwischen diesen Komponenten.

Abbildung 1: Architekturdiagramm



Quelle: Eigene Darstellung

Die Benutzeroberfläche wird im Browser ausgeführt und stellt den Spielzustand dar sowie die verfügbaren Benutzeraktionen bereit. Die Kommunikationsschicht bildet mit FastAPI die HTTP- und WebSocket-Schnittstellen ab und nimmt Nachrichten der Clients entgegen. HTTP wird für abgegrenzte Vorgänge wie das Erstellen, Beitreten und Starten einer Partie sowie für den initialen Abruf des Spielzustands verwendet. Für die fortlaufende Kommunikation in der Lobby und während einer

laufenden Partie wird anschließend eine WebSocket-Verbindung aufgebaut. Über diese werden Spielaktionen an den Server übertragen und Änderungen des Spielzustands an die verbundenen Clients verteilt. Die übertragenen Daten werden jeweils als JSON strukturiert.

Die Anwendungslogik koordiniert die vorhandenen Partien und vermittelt zwischen der Kommunikation mit den Clients und der eigentlichen Spiellogik. Die Spiellogik enthält die zentralen Geschäftsobjekte und Spielregeln und bleibt weitgehend unabhängig von der grafischen Benutzeroberfläche und der Netzwerkkommunikation. Eine zentrale Rolle innerhalb der Anwendungslogik übernimmt der GameManager. Er verwaltet die vorhandenen Partien und stellt die Verbindung zwischen der Kommunikationsschicht und den einzelnen Spielinstanzen her. Eine konkrete Partie wird durch die Klasse Game repräsentiert, die unter anderem Spieler, Spielzustand und Zugsteuerung verwaltet. Weitere Klassen wie Board und Target bilden die eigentlichen Bestandteile und Regeln des Spiels ab.

Auf eine Microservice-Architektur oder eine Aufteilung auf mehrere unabhängige Serverdienste wird bewusst verzichtet. Microservice-Architekturen können Vorteile hinsichtlich Skalierbarkeit, Resilienz und unabhängiger Bereitstellung einzelner Dienste bieten. Allerdings gehen sie mit zusätzlichem Aufwand für Kommunikation, Koordination, Sicherheit und Überwachung der einzelnen Dienste einher (Salah et al., 2016, S. 323–324). Da Mystic Maze nur einen begrenzten Funktionsumfang besitzt und hauptsächlich innerhalb eines lokalen Netzwerks verwendet werden soll, werden diese Vorteile nicht benötigt. Eine Verteilung auf mehrere Serverdienste würde die Entwicklung und Verwaltung der Anwendung komplizierter machen, ohne einen entsprechenden Nutzen für das Projekt zu bieten.

Die gewählte Architektur unterstützt gleichzeitig die Wartbarkeit und Testbarkeit der Anwendung. Da die Spiellogik von der Benutzeroberfläche und der Netzwerkkommunikation getrennt wird, können zentrale Spielregeln unabhängig vom Webbrowser getestet werden. Änderungen an der grafischen Darstellung sollen dadurch ebenfalls möglich sein, ohne die grundlegende Spiellogik verändern zu müssen.

4. Struktur

Die interne Struktur von Mystic Maze baut auf dem Datenmodell aus der Spezifikation auf. Dort werden die fachlichen Zusammenhänge zwischen Partie, Spieler, Spielfeld, Kachel und Ziel beschrieben. Im folgenden Kapitel werden die dazugehörigen Python-Klassen und deren Aufgaben genauer erläutert.

Die Verwaltung mehrerer laufender Partien übernimmt der GameManager. Er hält die vorhandenen Game-Instanzen vor und stellt Operationen zum Erstellen, Abrufen und Entfernen einer Partie sowie zum Hinzufügen und Abrufen von Spielern bereit. Jede Partie erhält eine eindeutige Partie-ID, über die eingehende Anfragen der richtigen Game-Instanz zugeordnet werden können.

Beim Beitritt erhält jeder Spieler zusätzlich einen festen Index innerhalb der Spielerliste. Dieser wird als player_index bezeichnet und dient zur Identifizierung des Spielers innerhalb einer Partie. Zusammen mit der Partie-ID kann die Kommunikationsschicht dadurch Anfragen und WebSocket-Verbindungen dem jeweiligen Spieler zuordnen.

Die Klasse Game bildet eine einzelne Partie ab und enthält den wesentlichen Spielzustand sowie die Steuerung des Spielablaufs. Dazu gehören unter anderem die teilnehmenden Spieler, das Spielfeld, der aktive Spieler, der aktuelle Zugstatus, bereits aktivierte Schlüssel sowie der Status der Partie. Game koordiniert außerdem zentrale Spielaktionen wie das Starten einer Partie, das Einschieben der freien Kachel, das Bewegen eines Spielers, die Verarbeitung von Spezialeffekten und das Beenden eines Spielzugs. Bereits in der bestehenden Spiellogik werden beispielsweise der Verschiebe- und Bewegungsstatus sowie der Gewinner innerhalb der Game-Instanz verwaltet.

Das eigentliche Spielfeld wird durch die Klasse Board repräsentiert. Sie verwaltet das 7 × 7-Kachelraster sowie die aktuell freie Kachel. Zu ihren Aufgaben gehören insbesondere die Manipulation des Spielfelds durch das Einschieben der freien Kachel und die Ermittlung erreichbarer Positionen. Die Entscheidung, ob eine konkrete Spielaktion gemäß dem aktuellen Zug erlaubt ist, verbleibt dagegen überwiegend bei Game. Dadurch wird die geometrische Spielfeldlogik von der übergeordneten Regel- und Zugsteuerung getrennt.

Eine einzelne Labyrinthkachel wird durch Tile dargestellt. Eine Kachel besitzt insbesondere einen Kacheltyp und eine Orientierung. Zusätzlich kann sie ein Ziel, ein Tor oder ein Portal enthalten. Die Ausprägungen von Kacheltypen, Richtungen und Toren werden durch Enums wie TileType, Direction und GateType abgebildet. Dadurch werden nur vordefinierte und fachlich zulässige Werte innerhalb der Spiellogik verwendet.

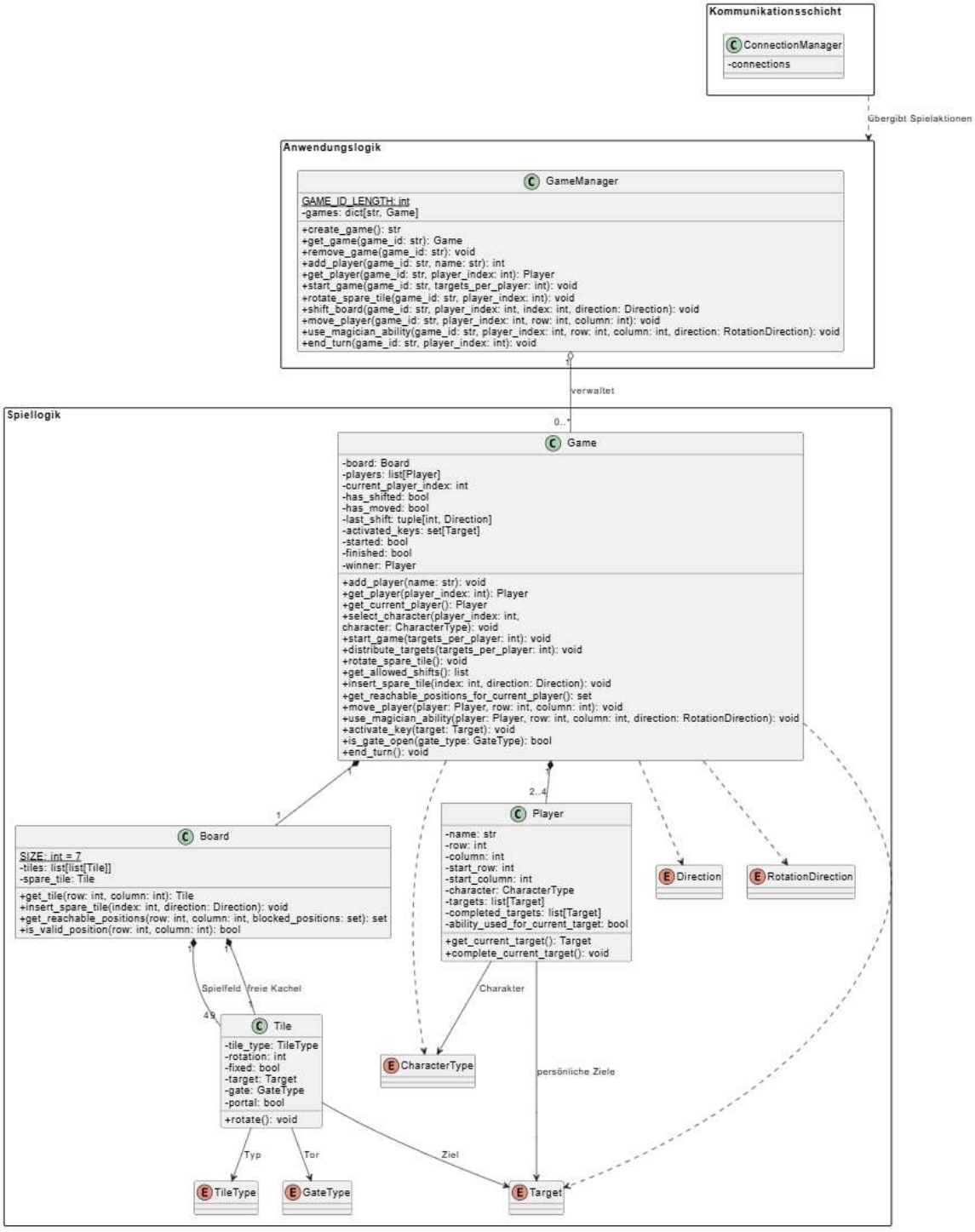
Die Klasse Player speichert die Informationen eines einzelnen Spielers. Dazu gehören der Name, die aktuelle Position, die Startposition, der gewählte Charakter sowie die noch offenen und bereits abgeschlossenen Ziele. Außerdem wird gespeichert, ob die Charakterfähigkeit für das aktuelle Ziel bereits verwendet wurde. Über die Methode get_current_target() kann das aktuell gesuchte Ziel abgerufen werden. Diese Information benötigt Game beispielsweise, um zu überprüfen, ob ein Spieler sein Ziel erreicht hat.

Die möglichen Ziele und Artefakte werden durch die Enumeration Target festgelegt. Anders als im fachlichen Datenmodell der Spezifikation werden Zielkarten und Artefakte nicht durch getrennte Klassen dargestellt. Stattdessen werden die Werte von Target sowohl den Spielern als persönliche Ziele als auch den Kacheln als auffindbare Objekte zugeordnet. Besondere Ziele wie die Schlüssel werden zusätzlich von Game verarbeitet. Wird ein Schlüssel aktiviert, verändert sich der Zustand des dazugehörigen Tores.

Für die Verwaltung der WebSocket-Verbindungen wird die Klasse ConnectionManager verwendet. Sie gehört zur Kommunikationsschicht und verwaltet die aktiven Verbindungen der einzelnen Spieler. Eingehende Spielaktionen werden von der Kommunikationsschicht an den GameManager weitergegeben. Dieser ordnet die Aktionen der entsprechenden Game-Instanz zu, in der die Spielregeln überprüft und die Aktionen verarbeitet werden. Anschließend wird der aktualisierte Spielzustand über die Kommunikationsschicht und den ConnectionManager an die verbundenen Clients übertragen. Dadurch bleiben die Verwaltung der Verbindungen und die eigentliche Spiellogik voneinander getrennt. Ergänzend zum ConnectionManager werden Hilfsmodule eingesetzt. Diese übernehmen unter anderem die Bereitstellung der HTTP- und WebSocket-Schnittstellen, die Verarbeitung eingehender Spielaktionen sowie die Verteilung aktualisierter Spielzustände an die verbundenen Clients.

Die Klassen sind im folgenden UML-Klassendiagramm dargestellt. Aus Gründen der Übersichtlichkeit werden dort ausschließlich die wesentlichen Methoden, die zur Spiellogik beitragen, aufgeführt. Hilfsmodule und interne Hilfsfunktionen werden nicht dargestellt. Die Hilfsmodule dienen der Trennung technischer Aufgaben, während die internen Hilfsfunktionen einzelne Verarbeitungsschritte strukturieren. Beide verändern die grundlegenden Beziehungen zwischen den dargestellten Klassen nicht.

Abbildung 2: UML-Klassendiagramm



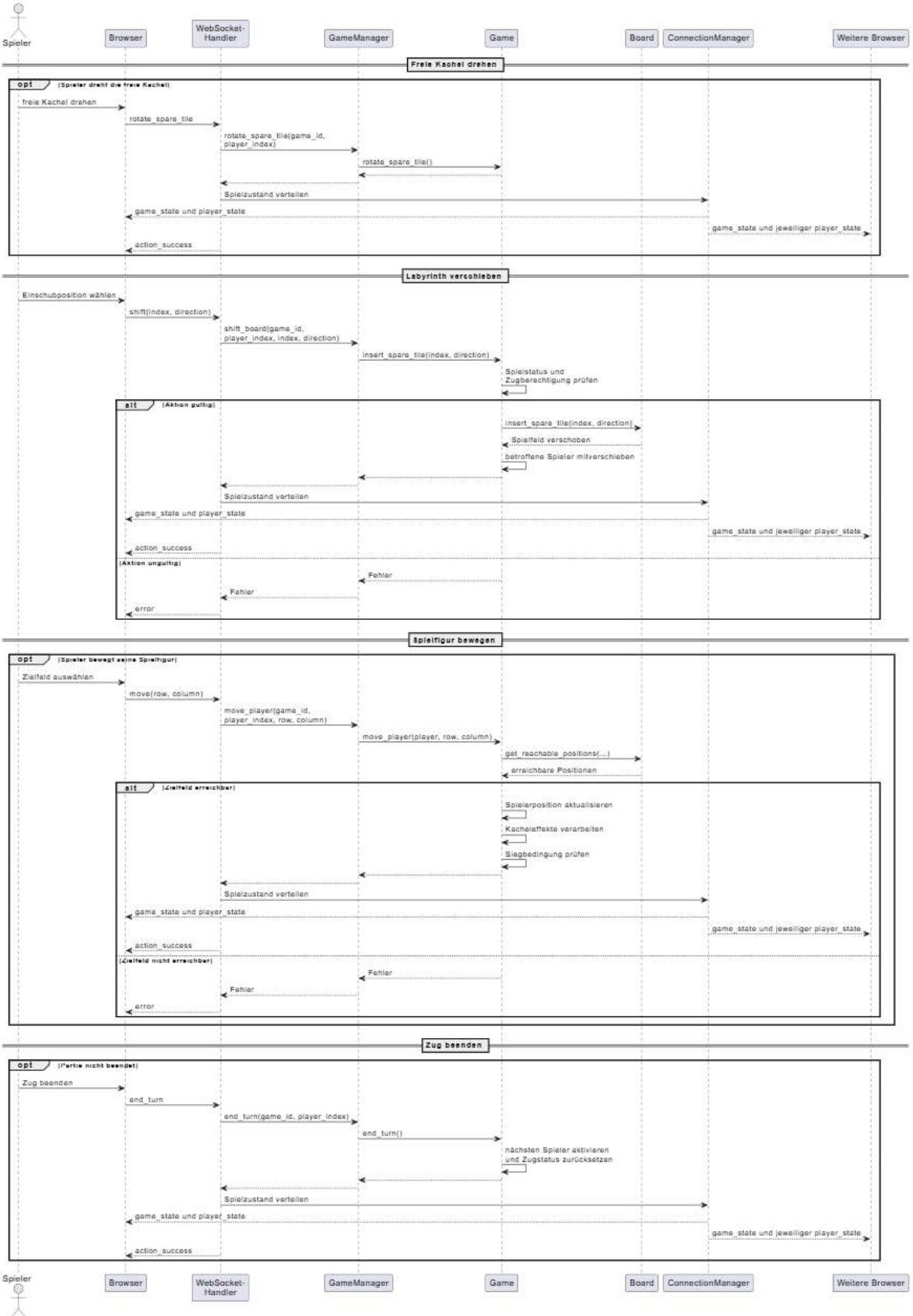
Quelle: Eigene Darstellung

Die dargestellte Struktur folgt der in Kapitel 3 beschriebenen Trennung der Verantwortlichkeiten. GameManager und ConnectionManager übernehmen übergeordnete Verwaltungs- beziehungsweise Kommunikationsaufgaben, während Game, Board, Tile und Player die eigentliche Spiellogik und den Spielzustand abbilden. Dadurch bleibt insbesondere die Spiellogik unabhängig von der konkreten Weboberfläche und kann separat automatisiert getestet werden.

5. Verhalten

Zur Visualisierung eines zentralen Systemablaufs wird der Ablauf eines Spielzugs als UML-Sequenzdiagramm dargestellt. Optionale Sonderaktionen wie die Verwendung einer Charakterfähigkeit werden aus Gründen der Übersichtlichkeit nicht einzeln dargestellt. Auch diese Aktionen werden an den Server übertragen, dort überprüft und verarbeitet. Anschließend wird der aktualisierte Spielzustand an die verbundenen Spieler gesendet.

Abbildung 3: Sequenzdiagramm Spielzug



Quelle: Eigene Darstellung

Das UML-Sequenzdiagramm zeigt den Ablauf eines Spielzugs von Mystic Maze und die Kommunikation zwischen den beteiligten Komponenten. Die Spielaktionen werden vom Browser über die WebSocket-Verbindung an den Server übertragen. Dort werden sie durch den GameManager der entsprechenden Game-Instanz zugeordnet. Game überprüft die Spielregeln und führt die Aktionen aus. Für das Verschieben des Spielfelds und die Ermittlung erreichbarer Positionen wird die Klasse Board verwendet.

Während eines Spielzugs kann der Spieler zunächst die freie Kachel drehen und muss anschließend das Spielfeld verschieben. Danach kann er seine Spielfigur bewegen oder den Zug direkt beenden. Nach jeder erfolgreichen Spielaktion wird der aktualisierte Spielzustand an die verbundenen Clients übertragen. Die Verteilung wird durch die Kommunikationsschicht koordiniert, wobei der ConnectionManager die aktiven WebSocket-Verbindungen verwaltet. Ungültige Aktionen werden nicht ausgeführt und der Spieler erhält eine Fehlermeldung.

Literaturverzeichnis

Balsamiq. (o. J.). *What is Balsamiq? Low-fidelity wireframing tool - Balsamiq Help Center*. Abgerufen 23. Oktober 2026, von https://balsamiq.com/support/getting-started/what-is-balsamiq/

Bray, T. (2017). The JavaScript Object Notation (JSON) Data Interchange Format. *Internet Engineering Task Force (IETF)*, 1–15. https://doi.org/10.17487/RFC8259

FastAPI. (o. J.-a). *Features - FastAPI*. Abgerufen 23. Oktober 2026, von https://fastapi.tiangolo.com/features/

FastAPI. (o. J.-b). *Run a Server Manually - FastAPI*. Abgerufen 23. Oktober 2026, von https://fastapi.tiangolo.com/deployment/manually/#use-the-fastapi-run-command

FastAPI. (o. J.-c). *WebSockets - FastAPI*. Abgerufen 23. Oktober 2026, von https://fastapi.tiangolo.com/advanced/websockets/

Fette, I., & Melnikov, A. (2011). The WebSocket Protocol. *Internet Engineering Task Force (IETF)*, 1–69. https://doi.org/10.17487/RFC6455

Fielding, R., Nottingham, M., & Reschke, J. (2022). HTTP Semantics. *Internet Engineering Task Force (IETF)*, 1–194. https://doi.org/10.17487/RFC9110

Garlan, D., & Shaw, M. (1993). *AN INTRODUCTION TO SOFTWARE ARCHITECTURE* (S. 1–39). https://doi.org/10.1142/9789812798039_0001

Git. (o. J.). *Git - user-manual Documentation*. Abgerufen 23. Oktober 2026, von https://git-scm.com/docs/user-manual

GitHub. (o. J.). *About repositories - GitHub Docs*. Abgerufen 23. Oktober 2026, von https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories

JetBrains. (o. J.-a). *AI Assistant in JetBrains IDEs | PyCharm Documentation*. Abgerufen 26. Oktober 2026, von https://www.jetbrains.com/help/pycharm/ai-assistant-in-jetbrains-ides.html

JetBrains. (o. J.-b). *Quick start guide | PyCharm Documentation*. Abgerufen 23. Oktober 2026, von https://www.jetbrains.com/help/pycharm/quick-start-guide.html

Mozilla. (o. J.). *Your first website - Learn web development | MDN*. Abgerufen 23. Oktober 2026, von https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Your_first_website

OpenAI. (o. J.). *Images in ChatGPT*. Abgerufen 29. Oktober 2026, von https://help.openai.com/en/articles/11084440-images-in-chatgpt

PlantUML. (o. J.). *Open-Source-Tool, das einfache Textbeschreibungen verwendet UML-Diagramme zu zeichnen.* Abgerufen 23. Oktober 2026, von https://plantuml.com/de/

pytest.org. (o. J.). *pytest documentation*. Abgerufen 23. Oktober 2026, von https://docs.pytest.org/en/latest/

Python Software Foundation. (o. J.-a). *PyPI Docs*. Abgerufen 23. Oktober 2026, von https://docs.pypi.org/

Python Software Foundation. (o. J.-b). *The Python Tutorial — Python 3.15.1 documentation*. Abgerufen 23. Oktober 2026, von https://docs.python.org/3/tutorial/

Salah, T., Jamal Zemerly, M., Chan Yeob Yeun, Al-Qutayri, M., & Al-Hammadi, Y. (2016). The evolution of distributed systems towards microservices architecture. *2016 11th International Conference for Internet Technology and Secured Transactions (ICITST)*, 318–325. https://doi.org/10.1109/ICITST.2016.7856721